Глубокое обучение с подкреплением

Модуль 1: основы и первый агент

Курс Deep RL от Hugging Face

Полное учебное издание на основе исходных MDX-файлов

Исходный материал: курс Deep RL от Hugging Face.

Содержание

1 Введение в глубокое обучение с подкреплением **2**

2 Что такое обучение с подкреплением? **3**

2.1 Общая идея . . . 3

3 Формальная схема обучения с подкреплением **5**

3.1 Процесс RL . . . 5

3.2 Гипотеза вознаграждения: центральная идея RL . . . 6

3.3 Марковское свойство . . . 6

3.4 Пространство наблюдений и состояний . . . 6

3.5 Пространство действий . . . 8

3.6 Вознаграждения и дисконтирование . . . 9

4 Типы задач **11**

4.1 Эпизодические задачи . . . 11

4.2 Продолжающиеся задачи . . . 12

5 Компромисс между исследованием и использованием знаний **13**

6 Два основных подхода к решению задач RL **15**

6.1 Политика π: «мозг» агента . . . 15

6.2 Методы на основе политики . . . 16

6.3 Методы на основе функции ценности . . . 18

7 Почему обучение с подкреплением называется «глубоким»? **20**

8 Краткое резюме **21**

9 Глоссарий **21**

10 Обучаем первого агента глубокого обучения с подкреплением **22**

11 Модуль 1: обучаем первого агента Deep RL **24**
11.1 Цели ноутбука . . . 24
11.2 Ноутбук курса Deep Reinforcement Learning . . . 25
11.3 Предварительные требования . . . 25
11.4 Краткое повторение Deep RL . . . 26

12 Обучим первого агента Deep RL и загрузим его в Hub **26**
12.1 Получение сертификата . . . 26
12.2 Настройка GPU . . . 27
12.3 Установка зависимостей и создание виртуального экрана . . . 27
12.4 Импорт библиотек . . . 28
12.5 Как работает Gymnasium . . . 29
12.6 Создание среды LunarLander и знакомство с ней . . . 30
12.7 Создание модели . . . 32
12.8 Обучение агента PPO . . . 33
12.9 Оценка качества агента . . . 33
12.10Публикация обученной модели в Hub . . . 34
12.11Загрузка сохранённой модели LunarLander из Hub . . . 37
12.12Дополнительные задания . . . 38
12.13Продолжайте учиться, у вас всё получится . . . 39

13 Тест для самопроверки **39**
13.1 Ответы и объяснения . . . 40

14 Заключение **42**

15 Дополнительная литература **43**
15.1 Глубокое обучение с подкреплением . . . 43
15.2 Gym . . . 43

1 Введение в глубокое обучение с подкреплением

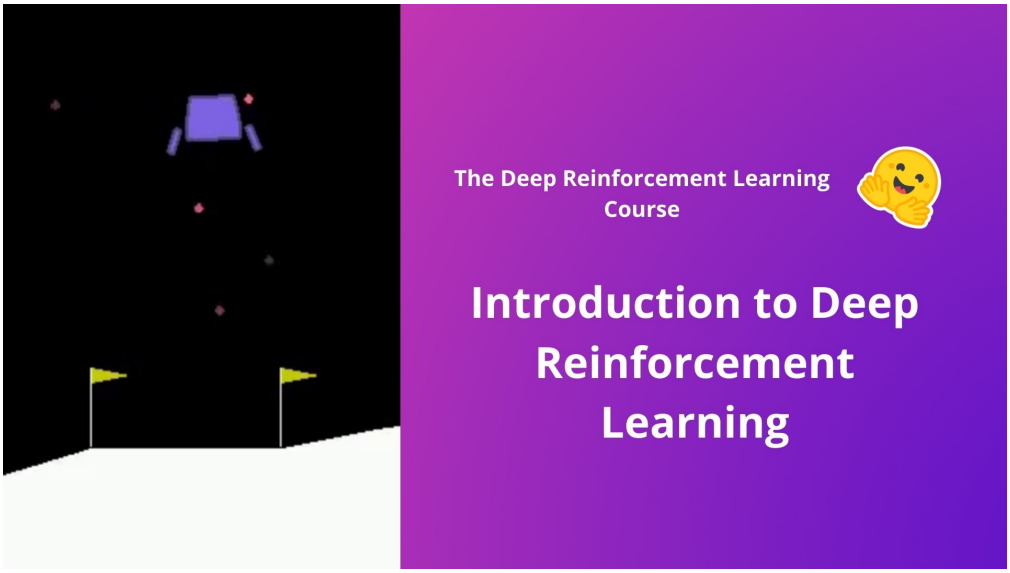


Рис. 1: Модуль 1: введение в глубокое обучение с подкреплением (официальная иллюстрация курса)

Добро пожаловать в одну из самых увлекательных областей искусственного интеллекта: **глубокое обучение с подкреплением.**

Глубокое обучение с подкреплением (Deep RL) — это разновидность машинного обучения, при которой агент учится **правильно действовать** в среде, **совершая действия** и **наблюдая их результаты.**

В этом первом модуле **вы изучите основы глубокого обучения с подкреплением.**

Затем с помощью Stable-Baselines3, библиотеки для глубокого обучения с подкреплением, вы **обучите агента, управляющего лунным посадочным модулем, правильно приземляться на Луну.**

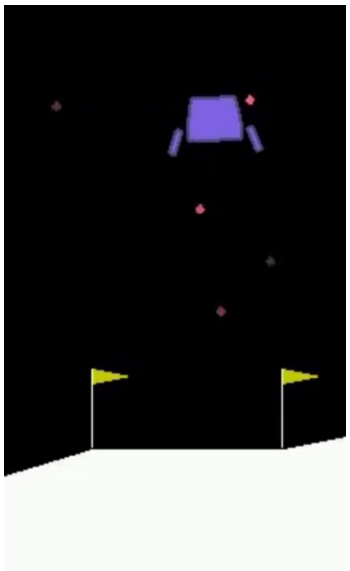


Рис. 2: Среда LunarLander: статичный кадр с официальной обложки курса; оригинальная анимация доступна по ссылке на GIF

И наконец, вы **загрузите обученного агента в Hugging Face Hub — бесплатную открытую платформу, где можно делиться моделями машинного обучения, наборами данных и демонстрациями.**

Прежде чем приступать к реализации агентов глубокого обучения с подкреплением, очень

важно **освоить эти понятия**. Цель этой главы — дать вам прочную теоретическую основу.

После завершения модуля в дополнительном разделе вы также **сможете обучить пса Huggy приносить палку и поиграть с ним.**



Рис. 3: Пёс Huggy: кадр из официальных материалов курса (оригинальное видео)

Приступим!

2 Что такое обучение с подкреплением?

Чтобы разобраться в обучении с подкреплением, сначала посмотрим на общую картину.

2.1 Общая идея

Основная идея обучения с подкреплением заключается в том, что агент (система искусственного интеллекта) учится в среде, **взаимодействуя с ней** методом проб и ошибок и **получая вознаграждения** (положительные или отрицательные) в качестве обратной связи за совершённые действия.

Обучение через взаимодействие со средой **знакомо нам по повседневному опыту.**

Представьте, например, что вы посадили младшего брата перед видеоигрой, в которую он никогда не играл, дали ему геймпад и оставили одного.

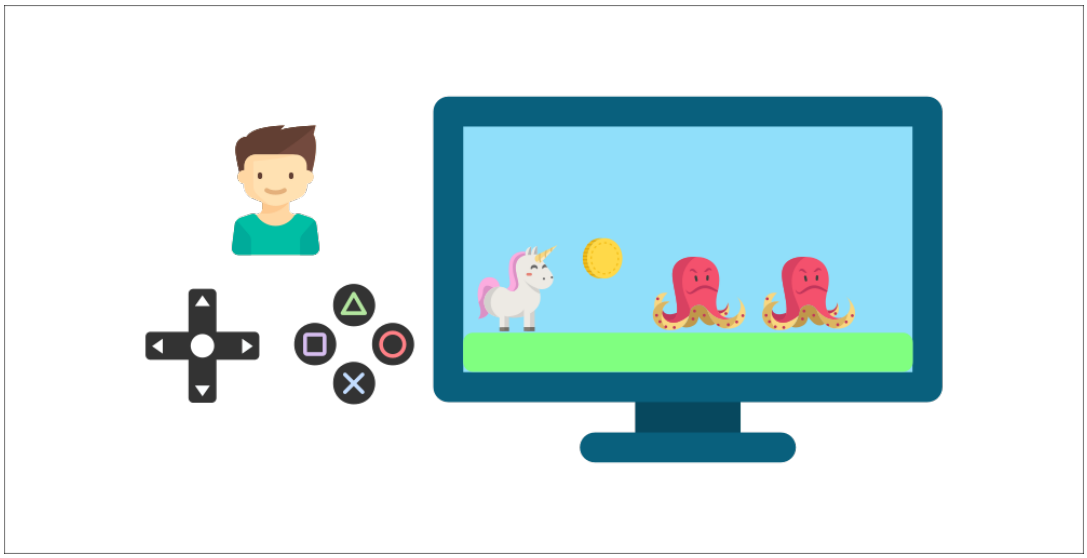


Рис. 4: Первое действие: игрок движется вправо

Ваш брат взаимодействует со средой (видеоигрой), нажимая кнопку движения вправо (действие). Он подбирает монету и получает вознаграждение +1. Это положительный результат: он только что понял, что в этой игре **нужно собирать монеты.**

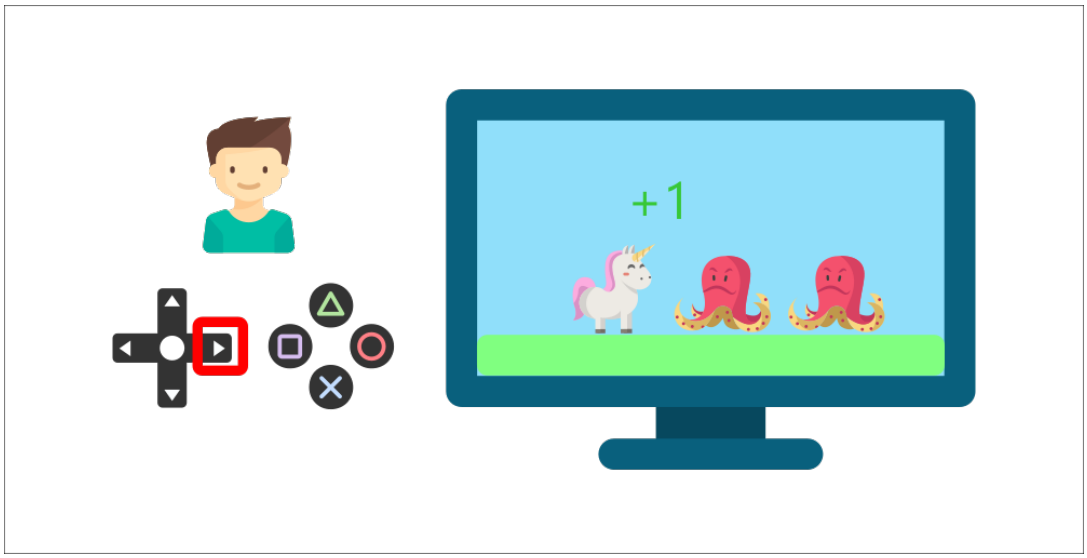


Рис. 5: Положительное вознаграждение: подбор монеты

Но затем **он снова нажимает кнопку вправо** и сталкивается с противником. Персонаж погибает, поэтому вознаграждение составляет -1.

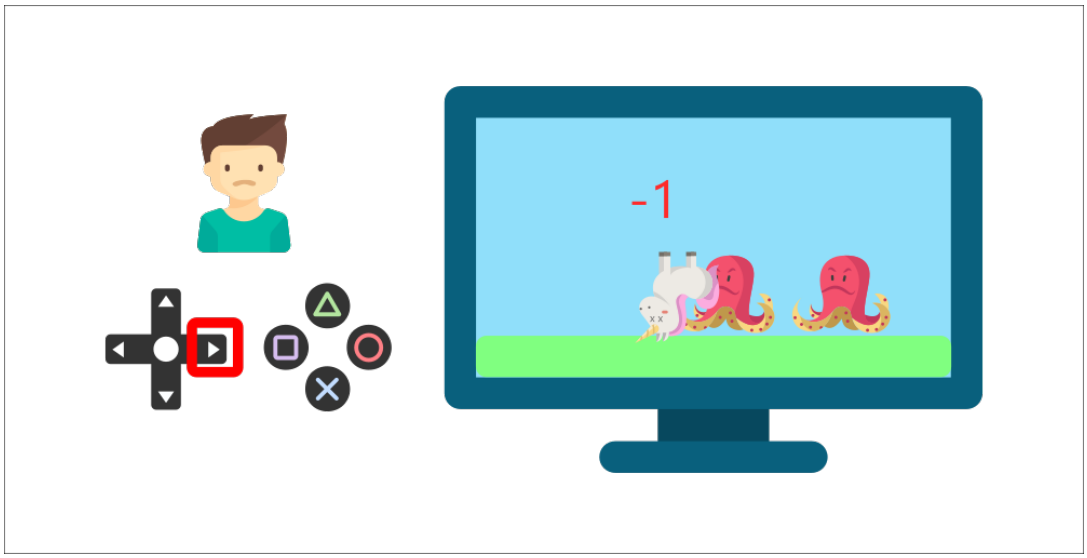


Рис. 6: Отрицательное вознаграждение: столкновение с противником

Взаимодействуя со средой методом проб и ошибок, ваш младший брат понимает, что **нужно собирать монеты, но избегать врагов.**

Без какого-либо внешнего руководства ребёнок постепенно начинает играть всё лучше.

Так же учатся люди и животные — **через взаимодействие.** Обучение с подкреплением — это **вычислительный подход к обучению на основе действий.**

Формальное определение

Теперь можно дать формальное определение:

Примечание к курсу. Обучение с подкреплением — это подход к решению задач управления (также называемых задачами принятия решений), в котором создаются агенты,

обучающиеся при взаимодействии со средой методом проб и ошибок и получающие в качестве единственной обратной связи вознаграждения — положительные или отрицательные.

Но как именно работает обучение с подкреплением?

3 Формальная схема обучения с подкреплением

3.1 Процесс RL

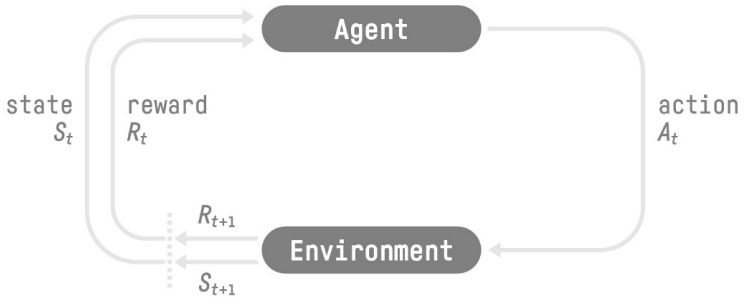


Рис. 7: Цикл RL: состояние, действие, вознаграждение и следующее состояние. Источник: Reinforcement Learning: An Introduction, Richard Sutton и Andrew G. Barto

Источник: http://incompleteideas.net/book/RLbook2020.pdf

Чтобы понять процесс RL, представим агента, который учится играть в платформер:

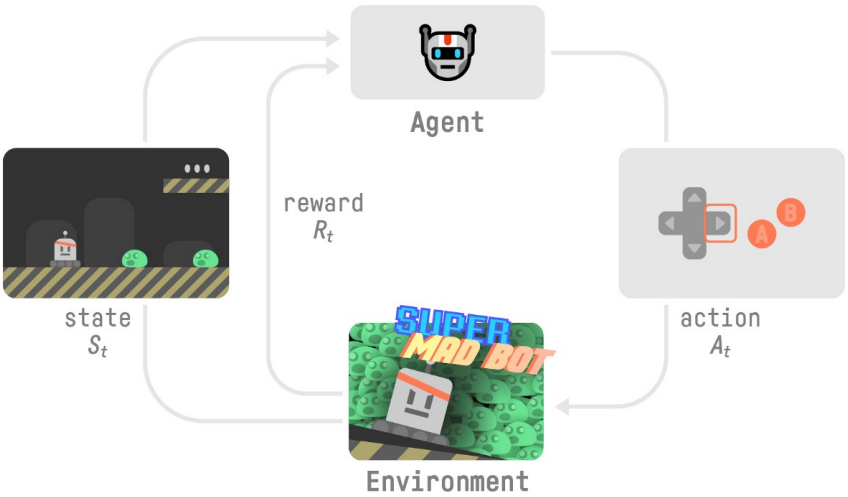


Рис. 8: Цикл взаимодействия в RL на примере видеоигры

- Наш агент получает от **среды состояние** S_0 — первый кадр игры.
- На основе этого **состояния** S_0 агент выбирает **действие** A_0 — движется вправо.
- Среда переходит в **новое состояние** S_1 — следующий кадр.

- Среда выдаёт агенту некоторое **вознаграждение** R_1 — мы не погибли, то есть получили *положительное вознаграждение +1*.

Такой цикл RL создаёт последовательность **состояний, действий, вознаграждений и следующих состояний.**

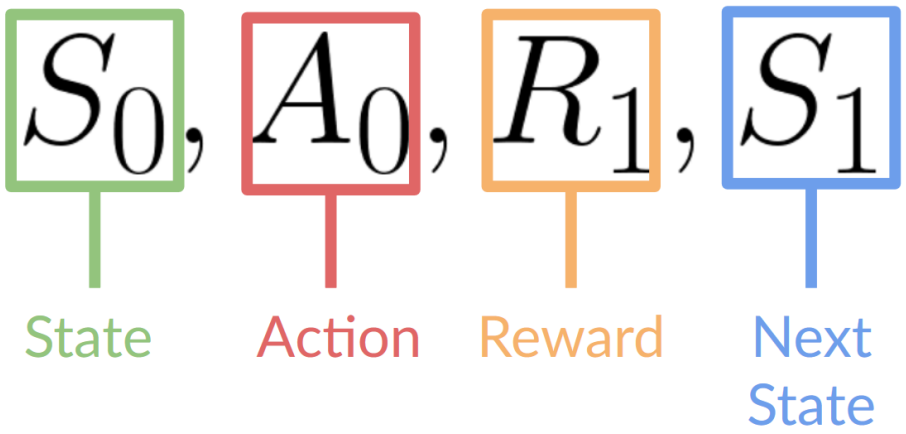


Рис. 9: Кортеж перехода: состояние, действие, вознаграждение, следующее состояние

Цель агента — *максимизировать* суммарное вознаграждение, **называемое ожидаемой отдачей (expected return).**

3.2 Гипотеза вознаграждения: центральная идея RL

⇒ Почему агент должен максимизировать ожидаемую отдачу?

Потому что RL основывается на **гипотезе вознаграждения**: любую цель можно описать как **максимизацию ожидаемой отдачи** (ожидаемого суммарного вознаграждения).

Поэтому, чтобы **получить наилучшее поведение**, в RL мы учимся выбирать действия, **максимизирующие ожидаемое суммарное вознаграждение.**

3.3 Марковское свойство

В научных работах процесс RL часто описывается как **марковский процесс принятия решений** (Markov Decision Process, MDP).

К марковскому свойству мы ещё вернёмся в следующих модулях. Пока достаточно запомнить: благодаря ему для **выбора действия агенту нужно знать только текущее состояние**, а **не всю историю предыдущих состояний и действий.**

3.4 Пространство наблюдений и состояний

Наблюдения и состояния — это **информация, которую агент получает от среды.** Для видеоигры такой информацией может быть кадр (скриншот), а для торгового агента — цена определённой акции.

Однако между *наблюдением* и *состоянием* существует различие:

- *Состояние s* — это **полное описание мира** (без скрытой информации). Оно доступно в полностью наблюдаемой среде.



Рис. 10: В шахматах мы получаем от среды состояние, поскольку имеем доступ ко всей информации на доске

В шахматах нам известна вся расстановка фигур, поэтому среда предоставляет состояние. Иными словами, она полностью наблюдаема.

- *Наблюдение o* — это **частичное описание состояния**. Оно доступно в частично наблюдаемой среде.



Рис. 11: В Super Mario Bros видна только часть уровня рядом с персонажем, поэтому агент получает наблюдение

В Super Mario Bros мы видим только ближайший к персонажу участок уровня, поэтому получаем наблюдение.

Таким образом, Super Mario Bros — частично наблюдаемая среда: мы получаем наблюдение, **поскольку видим только часть уровня.**

Примечание к курсу. В этом курсе слово «состояние» используется и для полного состояния, и для наблюдения, однако при реализации алгоритмов мы будем различать эти понятия.

Подведём итог:

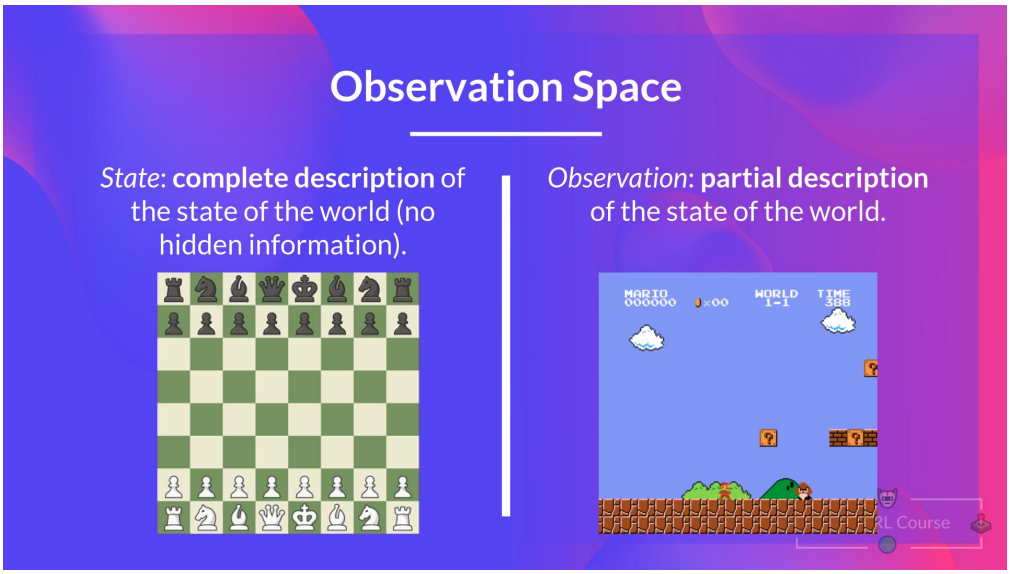


Рис. 12: Полное состояние и частичное наблюдение

3.5 Пространство действий

Пространство действий — множество **всех возможных действий в данной среде.**

Действия могут принадлежать *дискретному* или *непрерывному пространству*:

- *Дискретное пространство*: число возможных действий **конечно**.



Рис. 13: В Super Mario Bros всего четыре возможных действия: влево, вправо, вверх (прыжок) и вниз (приседание)

В Super Mario Bros множество действий конечно, поскольку доступны только четыре направления.

- *Непрерывное пространство*: число возможных действий **бесконечно**.



Рис. 14: У агента, управляющего автомобилем, бесконечное множество возможных действий: повернуть на 20°, 21,1°, 21,2°, подать звуковой сигнал, повернуть вправо на 20° и т. д.

Подведём итог:



Рис. 15: Дискретные и непрерывные пространства действий

Эту информацию важно учитывать, поскольку она **влияет на выбор алгоритма RL в дальнейшем.**

3.6 Вознаграждения и дисконтирование

Вознаграждение играет фундаментальную роль в RL, поскольку это **единственная обратная связь** для агента. Благодаря ему агент понимает, **насколько удачным было совершённое действие.**

Суммарное вознаграждение для момента времени **t** можно записать так:

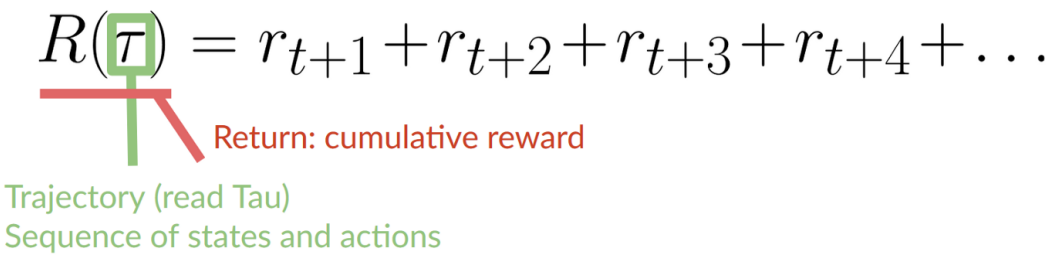


Рис. 16: Суммарное вознаграждение равно сумме всех вознаграждений в последовательности

Что эквивалентно выражению:

$$R(\tau) = \sum_{k=0}^{\infty} r_{t+k+1}$$

Рис. 17: Суммарное вознаграждение: $r_{t+1} + r_{t+2} + \cdots$, где r_{t+k+1} — вознаграждение на соответствующем шаге

Однако в реальности **нельзя просто складывать все вознаграждения таким образом.** Более ранние вознаграждения (в начале игры) **с большей вероятностью будут получены**, поскольку они предсказуемее отдалённых будущих вознаграждений.

Представим, что агент — маленькая мышь, которая за один шаг может переместиться на одну клетку, а её противник — кот (который тоже движется). Цель мыши — **съесть как можно больше сыра до того, как её поймает кот.**

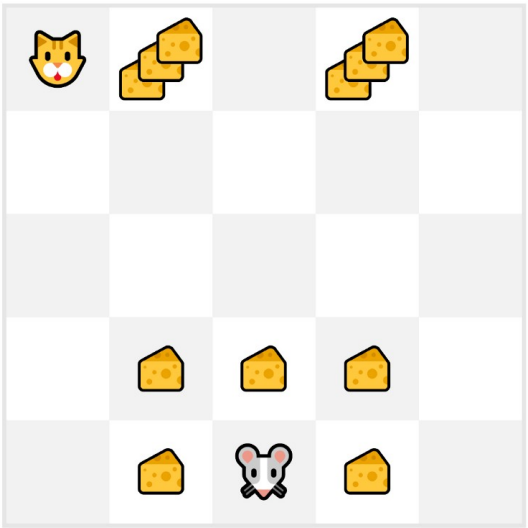


Рис. 18: Мышь, сыр и кот: иллюстрация отложенных вознаграждений

Как видно на схеме, **съесть сыр поблизости вероятнее, чем сыр рядом с котом** (чем ближе мышь к коту, тем опаснее).

Поэтому **вознаграждение рядом с котом, даже если оно больше (сыра там больше), будет сильнее дисконтироваться**: мы не уверены, что сможем до него добраться.

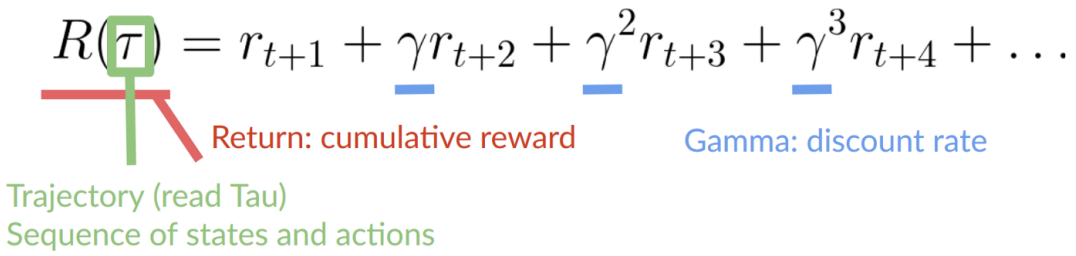
Для дисконтирования вознаграждений действуем следующим образом:

1. Вводим коэффициент дисконтирования γ. Он **должен лежать между 0 и 1**; чаще всего используют значения **от 0,95 до 0,99**.

- Чем больше γ, тем слабее дисконтирование. Это означает, что агент **сильнее учитывает долгосрочное вознаграждение.**
- Напротив, чем меньше γ, тем сильнее дисконтирование, то есть агент **больше ориентируется на ближайшее вознаграждение (ближний сыр).**

2. Затем каждое вознаграждение умножается на γ, возведённую в степень, соответствующую номеру шага. Чем дальше шаг во времени, тем ближе к мыши может оказаться кот, **поэтому вероятность получить будущее вознаграждение всё ниже.**

Наша ожидаемая дисконтированная суммарная отдача имеет вид:



$$R(\tau) = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

Рис. 19: Дисконтированная отдача с коэффициентом дисконтирования γ

4 Типы задач

Задача — это **конкретный экземпляр** проблемы RL. Различают два типа задач: **эпизодические** и **непрерывные**.

4.1 Эпизодические задачи

В этом случае есть начальная и конечная точки **(терминальное состояние). Вместе они задают эпизод**: последовательность состояний, действий, вознаграждений и новых состояний.

Например, в Super Mario Bros эпизод начинается при запуске уровня и заканчивается, **когда персонаж погибает или достигает конца уровня.**



Рис. 20: Начало нового эпизода

4.2 Продолжающиеся задачи

Это задачи, которые продолжаются бесконечно (**без терминального состояния**). Здесь агент должен **научиться выбирать лучшие действия, одновременно продолжая взаимодействовать со средой.**

Например, агент автоматической торговли акциями: у такой задачи нет заранее заданного конца. **Агент работает, пока мы сами не остановим его.**



Рис. 21: Биржевая торговля как пример продолжающейся задачи

Подведём итог:

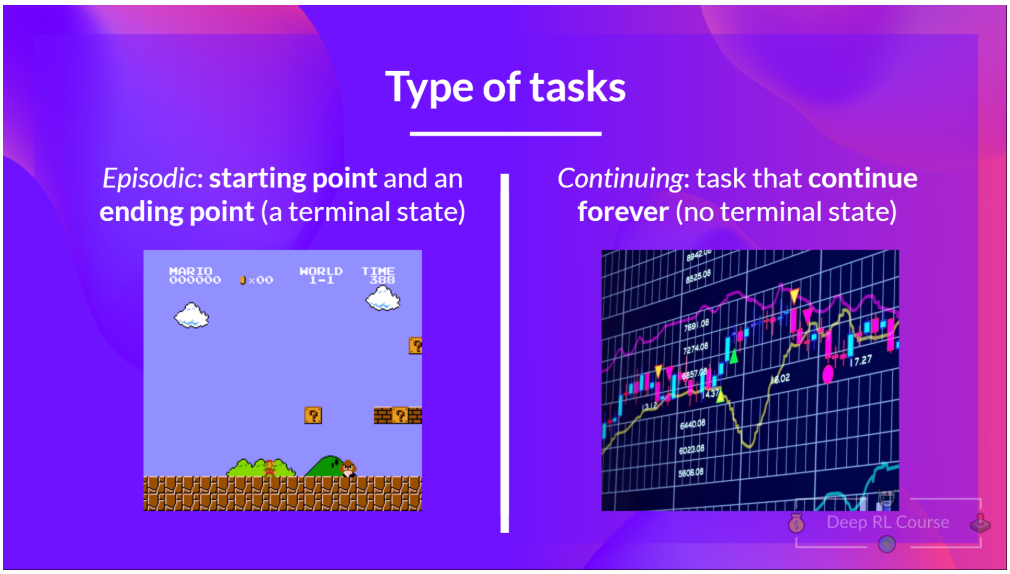


Рис. 22: Эпизодические и продолжающиеся задачи

5 Компромисс между исследованием и использованием знаний

Прежде чем рассматривать способы решения задач RL, необходимо обсудить ещё одну очень важную тему: *компромисс между исследованием (exploration) и использованием уже полученных знаний (exploitation).*

- *Исследование* — изучение среды с помощью случайных действий для **получения дополнительной информации о ней.**
- *Использование знаний* — **применение уже известной информации для максимизации вознаграждения.**

Помните, что цель агента — максимизировать ожидаемое суммарное вознаграждение. Однако **здесь легко попасть в распространённую ловушку**.

Рассмотрим пример:

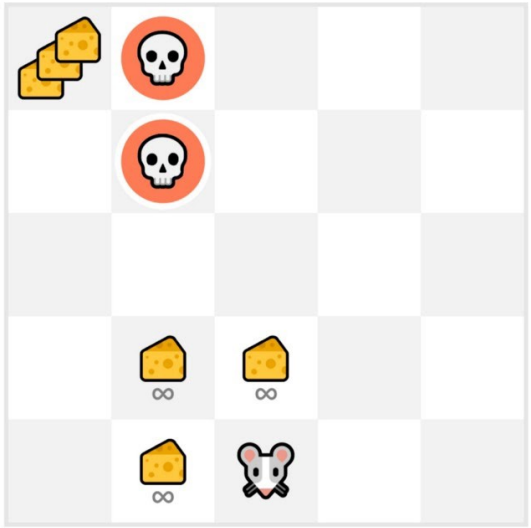


Рис. 23: Сырный лабиринт: локальные и отдалённые вознаграждения при выборе между исследованием и использованием знаний

В этой игре мышь может получать **неограниченное количество маленьких кусочков сыра** (по +1 каждый). Но в верхней части лабиринта находится огромная куча сыра (+1000).

Однако если заниматься только использованием известных возможностей, агент никогда не доберётся до огромной кучи. Он будет получать **ближайшие вознаграждения**, даже если они совсем небольшие (exploitation).

Если же агент немного исследует среду, он сможет **обнаружить большое вознаграждение** (огромную кучу сыра).

Именно это и называют компромиссом exploration/exploitation. Нужно соблюдать баланс между тем, насколько активно мы **исследуем среду** и насколько **используем уже известную информацию о ней.**

Поэтому необходимо **задать правило управления этим компромиссом**. В следующих модулях мы рассмотрим разные способы его реализации.

Если идея пока неочевидна, **вспомните повседневную задачу — выбор ресторана**:

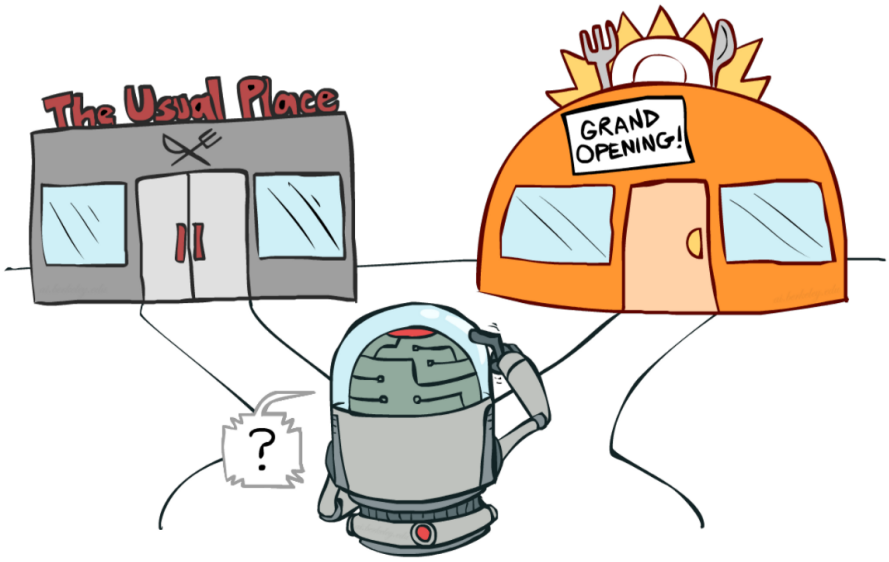


Рис. 24: Исследование и использование знаний: знакомый ресторан или неизвестная альтернатива. Источник: курс Berkeley AI

Источник: https://inst.eecs.berkeley.edu/~cs188/sp20/assets/lecture/lec15_6up.pdf

- *Использование знаний*: каждый день ходить в уже проверенное хорошее место, **рискуя не найти другой ресторан, который мог бы понравиться ещё больше.**
- *Исследование*: пробовать новые рестораны, рискуя получить плохой опыт, **но сохраняя шанс открыть отличное место.**

Подведём итог:

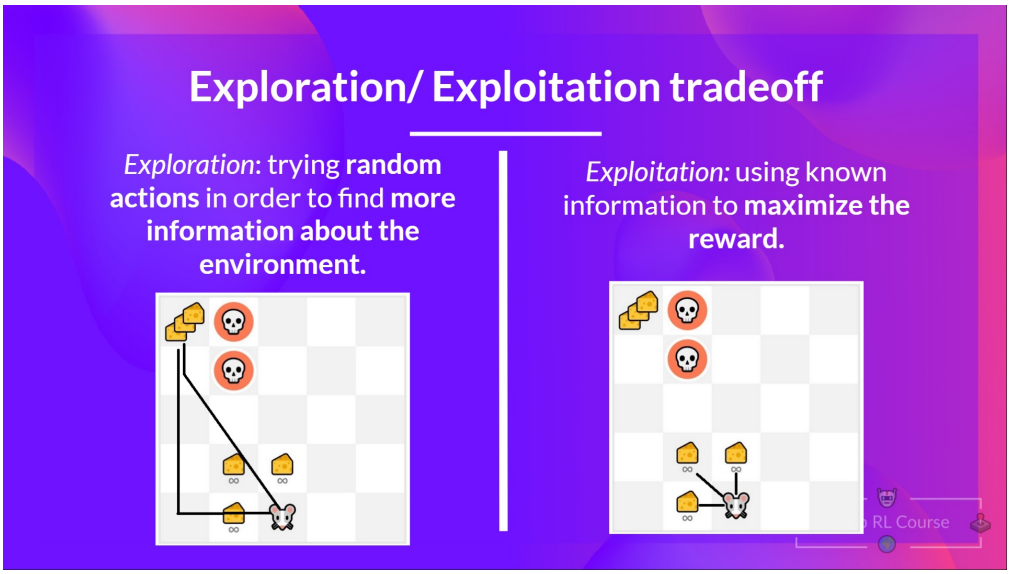


Рис. 25: Исследование и использование знаний: обобщающая схема

6 Два основных подхода к решению задач RL

Примечание к курсу. Теперь, когда мы познакомились с основами RL, как же решать соответствующие задачи?

Другими словами, как построить агента, способного **выбирать действия, максимизирующие его ожидаемое суммарное вознаграждение?**

6.1 Политика π: «мозг» агента

Политика π — это **«мозг» агента**, функция, определяющая, **какое действие нужно выбрать в данном состоянии.** Таким образом, она **задаёт поведение агента** в каждый момент времени.

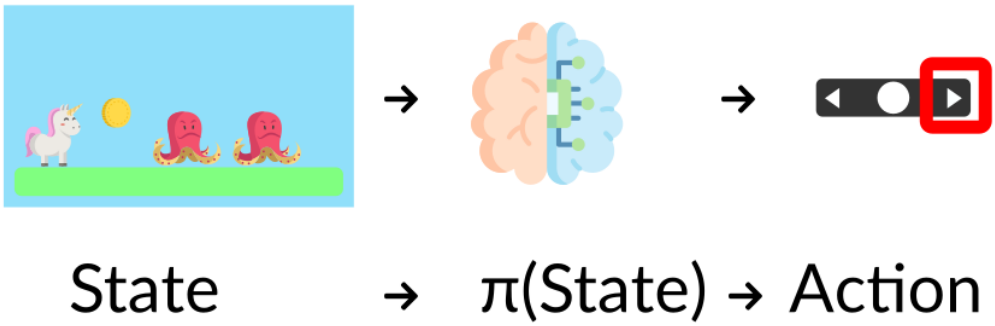


Рис. 26: Политика как «мозг» агента: функция, определяющая действие по текущему состоянию

Именно эту **функцию политики нам необходимо выучить**. Наша цель — найти оптимальную политику π^*, которая **максимизирует ожидаемую отдачу**, если агент ей следует. Оптимальную политику π^* мы находим **в процессе обучения.**

Существует два подхода к обучению агента для поиска оптимальной политики π^*:

- **Непосредственно** обучать агента выбирать **действие по текущему состоянию**: **методы на основе политики (Policy-Based Methods).**

- Косвенно: **обучать агента оценивать ценность состояний**, чтобы затем выбирать действия, **ведущие к более ценным состояниям**: методы на основе функции ценности (Value-Based Methods).

6.2 Методы на основе политики

В методах на основе политики **мы непосредственно обучаем функцию политики.**

Эта функция сопоставляет каждому состоянию наиболее подходящее действие. Другой вариант — определять **распределение вероятностей по возможным действиям в данном состоянии.**

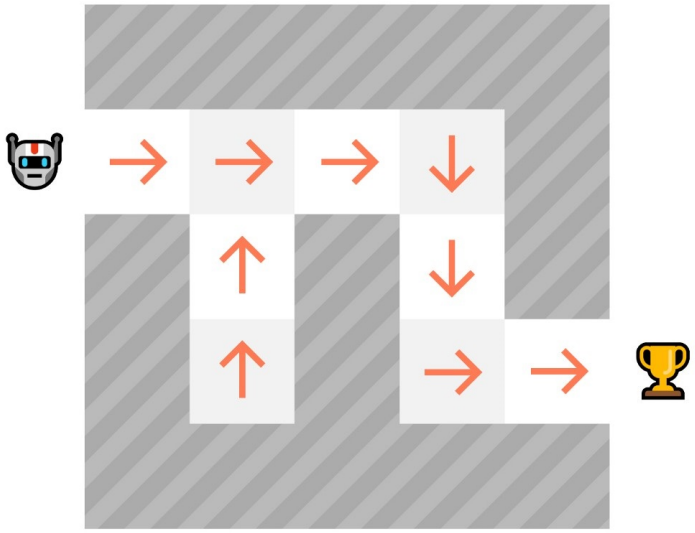


Рис. 27: Детерминированная политика непосредственно указывает действие на каждом шаге

Различают два вида политик:

- *Детерминированная*: в заданном состоянии политика **всегда возвращает одно и то же действие.**

$$a = \pi(s)$$

Рис. 28: Действие = политика(состояние)

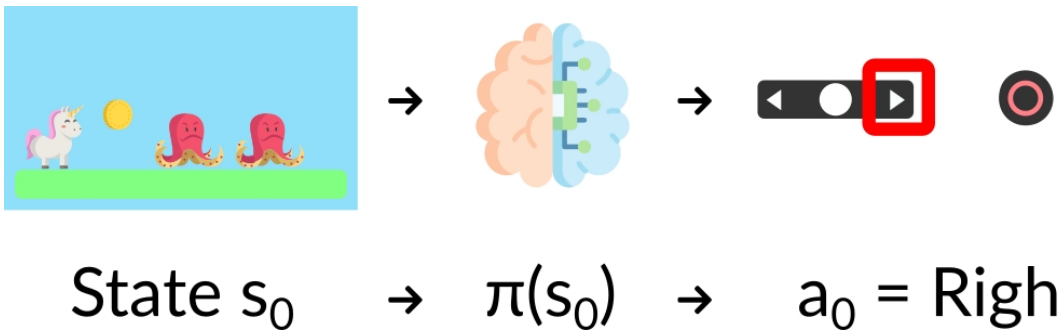


Рис. 29: Стохастическая политика: действия выбираются из условного распределения вероятностей

- *Стохастическая*: возвращает **распределение вероятностей по действиям.**

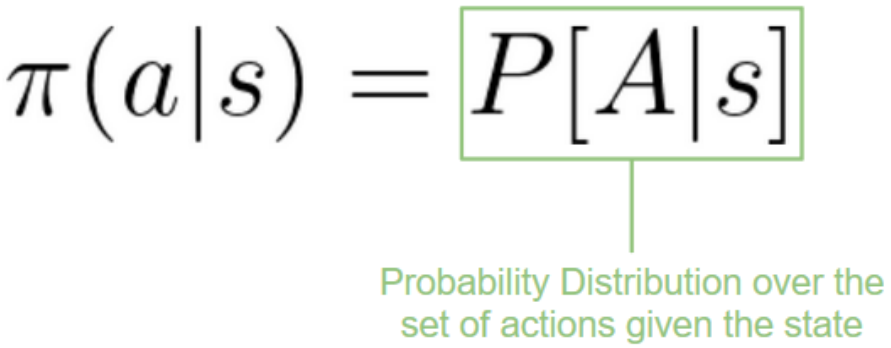


Рис. 30: Политика(действия | состояние) = распределение вероятностей возможных действий при текущем состоянии

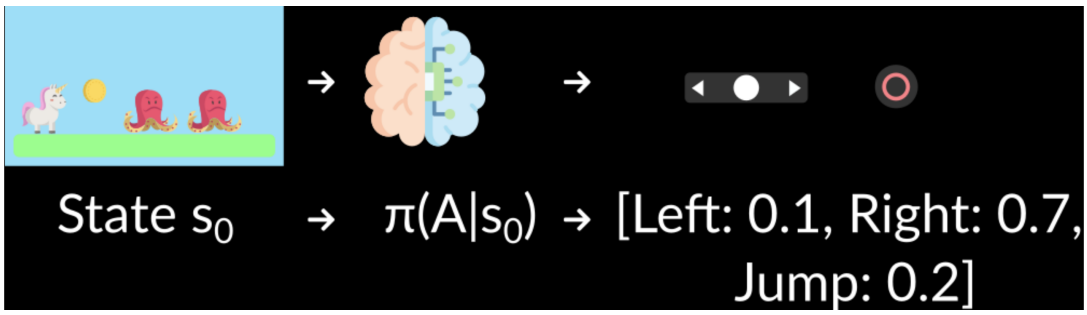


Рис. 31: Для заданного начального состояния стохастическая политика возвращает распределение вероятностей всех доступных действий

Подведём итог:

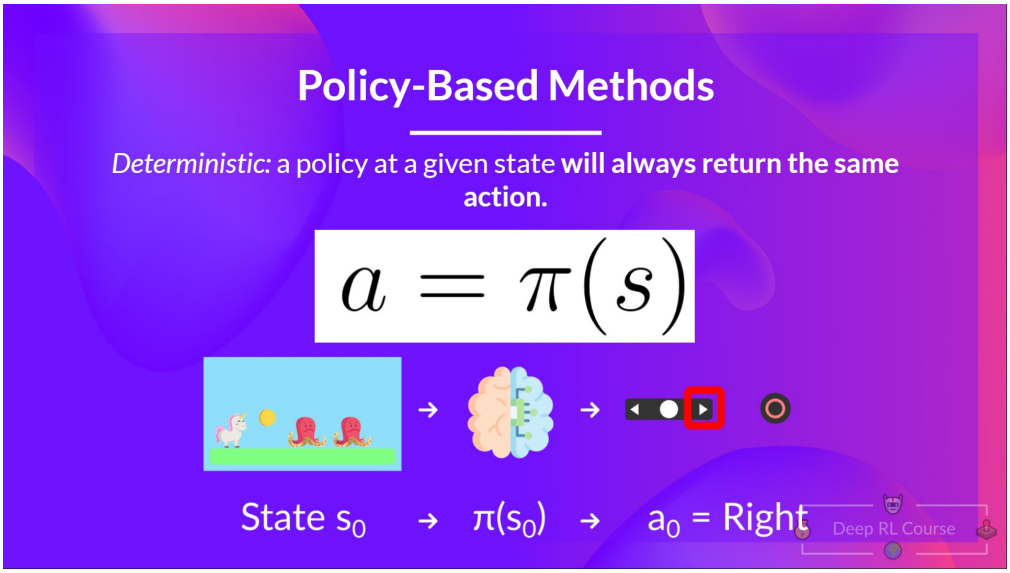


Рис. 32: Методы на основе политики: детерминированная политика

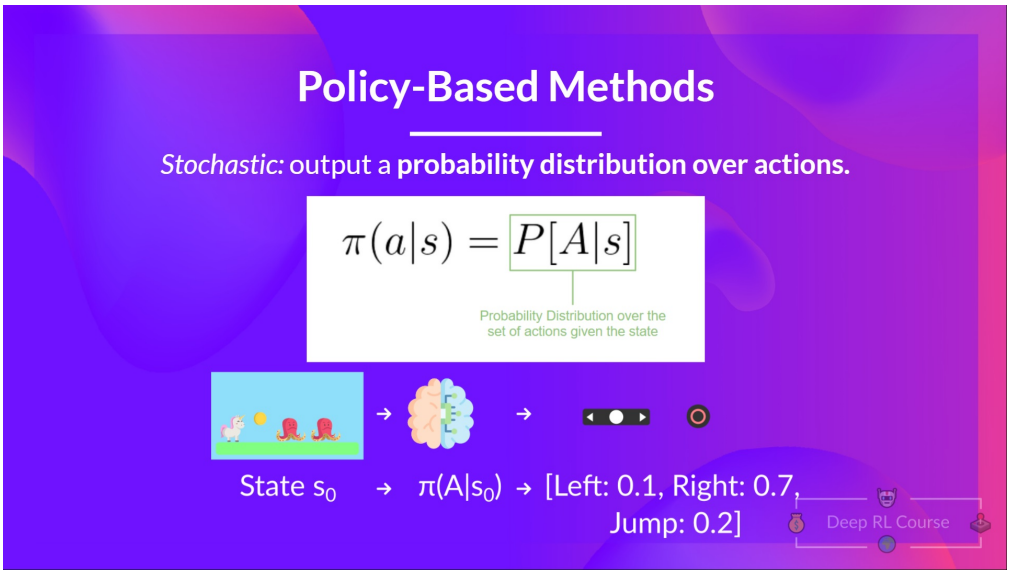


Рис. 33: Методы на основе политики: стохастическая политика

6.3 Методы на основе функции ценности

В методах на основе функции ценности вместо непосредственного обучения политики **мы обучаем функцию ценности**, которая сопоставляет состоянию ожидаемую **ценность нахождения в нём.**

Ценность состояния — это **ожидаемая дисконтированная отдача**, которую агент может получить, **начав из этого состояния и в дальнейшем следуя нашей политике.**

Фраза «следуя нашей политике» здесь означает, что она будет **«выбирать состояние с наибольшей ценностью».**

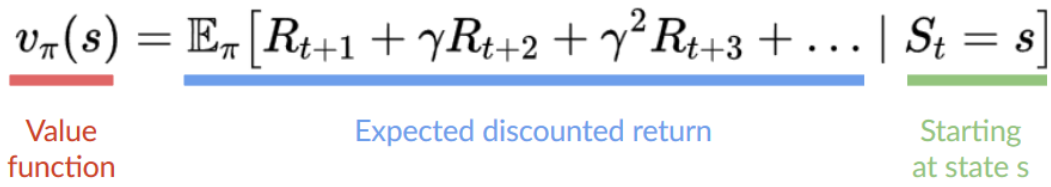


Рис. 34: Функция ценности состояния и ожидаемая дисконтированная отдача

На рисунке видно, что наша функция ценности **назначила значения всем возможным состояниям.**

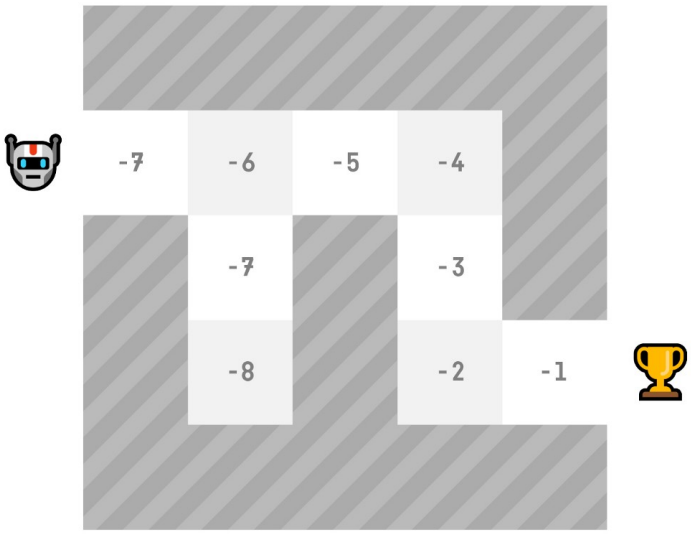


Рис. 35: Благодаря функции ценности политика на каждом шаге выбирает состояние с наибольшей оценкой: -7, затем -6, затем -5 и так далее, пока не достигнет цели

Благодаря функции ценности политика на каждом шаге выбирает состояние с наибольшей оценкой: -7, затем -6, затем -5 и так далее, пока агент не достигнет цели.

Подведём итог:

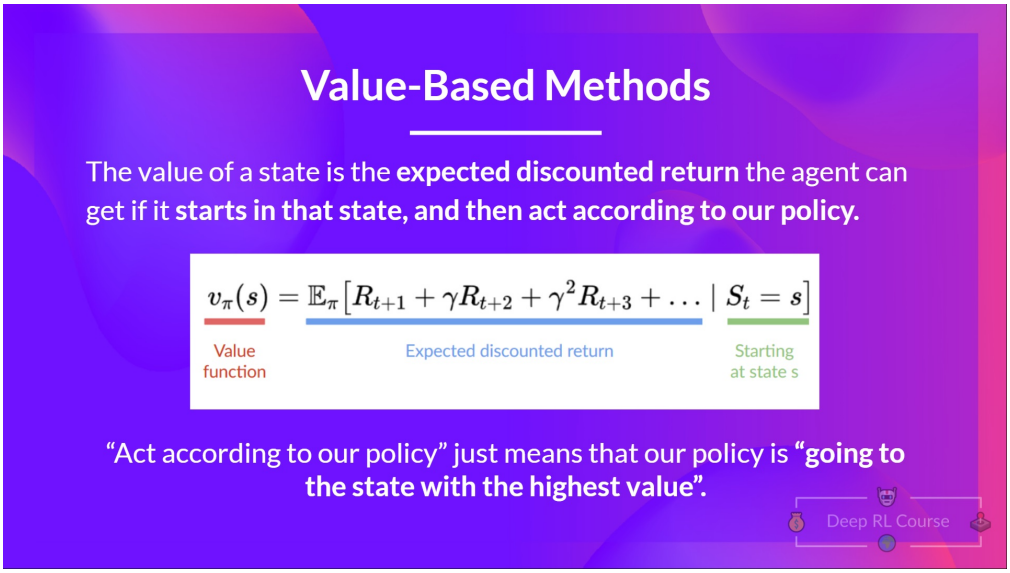


Рис. 36: Обучение с подкреплением на основе ценности: обзор функции ценности

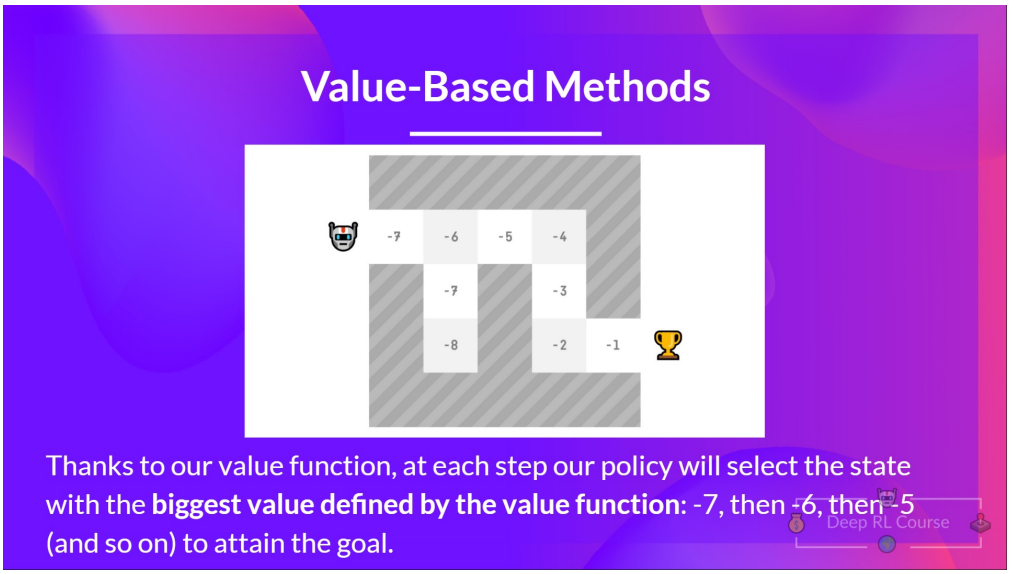


Рис. 37: Методы на основе функции ценности: выбор действий по ценности состояний

7 Почему обучение с подкреплением называется «глубоким»?

Примечание к курсу. До сих пор мы говорили об обучении с подкреплением. Но откуда в названии взялось слово «глубокое»?

Глубокое обучение с подкреплением предполагает **использование глубоких нейронных сетей для решения задач RL** — отсюда и название.

Например, в следующем модуле мы изучим два алгоритма на основе функции ценности: Q-Learning (классическое RL), а затем Deep Q-Learning.

Вы увидите, что в первом случае **мы применяем традиционный алгоритм**, чтобы создать Q-таблицу, которая подсказывает действие для каждого состояния.

Во втором подходе **используется нейронная сеть** для аппроксимации Q-значений.

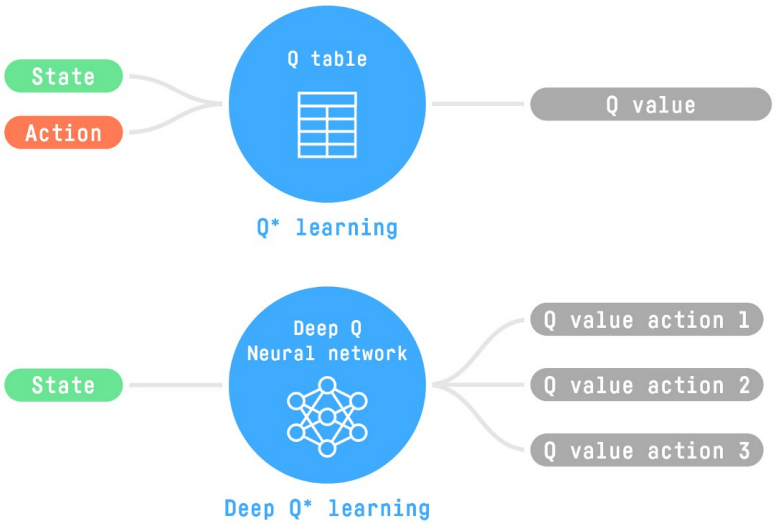


Рис. 38: Схема по мотивам учебного ноутбука Udacity по Q-Learning

Если вы ещё не знакомы с глубоким обучением, рекомендую посмотреть бесплатный курс FastAI Practical Deep Learning for Coders.

8 Краткое резюме

Мы рассмотрели немало информации! Подведём итоги:

- Обучение с подкреплением — вычислительный подход к обучению на основе действий. Мы создаём агента, который учится **взаимодействовать со средой методом проб и ошибок** и получает вознаграждения (отрицательные или положительные) в качестве обратной связи.
- Цель любого RL-агента — максимизировать ожидаемое суммарное вознаграждение (ожидаемую отдачу). В основе RL лежит **гипотеза вознаграждения**: **любую цель можно описать как максимизацию ожидаемого суммарного вознаграждения.**
- Процесс RL — цикл, порождающий последовательность **«состояние, действие, вознаграждение, следующее состояние».**
- Чтобы рассчитать ожидаемую суммарную отдачу, мы дисконтируем вознаграждения: ближайшие по времени вознаграждения (в начале игры) **вероятнее получить, поскольку их проще предсказать, чем отдалённые будущие вознаграждения.**
- Для решения задачи RL нужно **найти оптимальную политику**. Политика — «мозг» агента, который определяет, **какое действие совершить в текущем состоянии**. Оптимальная политика **выбирает действия, максимизирующие ожидаемую отдачу.**
- Оптимальную политику можно искать двумя способами:

  1. Непосредственно обучать политику: **методы на основе политики.**
  2. Обучать функцию ценности, оценивающую ожидаемую отдачу в каждом состоянии, и использовать её для выбора политики: **методы на основе функции ценности.**

- Наконец, слово «глубокое» означает использование **глубоких нейронных сетей для оценки выбираемого действия (методы на основе политики) или ценности состояния (методы на основе функции ценности).**

9 Глоссарий

Этот глоссарий создан сообществом. Мы приветствуем дополнения!

Агент

Агент учится **принимать решения методом проб и ошибок, получая от среды вознаграждения и наказания.**

Среда

Среда — это моделируемый мир, **в котором агент может учиться, взаимодействуя с ним.**

Марковское свойство

Оно означает, что выбранное агентом действие **зависит только от текущего состояния и не зависит от предыдущих состояний и действий.**

Наблюдения и состояния

- **Состояние**: полное описание состояния мира.
- **Наблюдение**: частичное описание состояния среды или мира.

Действия

- **Дискретные действия**: конечное число действий, например влево, вправо, вверх и вниз.
- **Непрерывные действия**: бесконечное число вариантов. Например, при управлении беспилотным автомобилем возможно бесконечно много управляющих воздействий.

Вознаграждение и дисконтирование

- **Вознаграждение**: фундаментальный элемент RL; сообщает агенту, было ли действие хорошим или плохим.
- Алгоритмы RL ориентированы на максимизацию **суммарного вознаграждения**.
- **Гипотеза вознаграждения**: задачи RL можно сформулировать как максимизацию (суммарной) отдачи.
- **Дисконтирование** применяется потому, что более ранние вознаграждения вероятнее получить: их проще предсказать, чем долгосрочные.

Типы задач

- **Эпизодическая**: имеет начальную и конечную точки.
- **Продолжающаяся**: имеет начало, но не имеет конечной точки.

Компромисс между исследованием и использованием знаний

- **Исследование**: выполнение случайных действий для изучения среды с получением обратной связи или вознаграждений.
- **Использование знаний**: применение известных сведений о среде для получения максимального вознаграждения.
- **Компромисс исследования и использования**: выбор баланса между **исследованием** среды и **использованием** уже имеющихся знаний.

Политика

- **Политика**: «мозг» агента, указывающий действие для заданного состояния.
- **Оптимальная политика**: политика, **максимизирующая ожидаемую отдачу**, когда агент ей следует. Находится в процессе *обучения*.

Методы на основе политики

- Один из подходов к решению задач RL.
- В нём политика обучается непосредственно.
- Она сопоставляет каждому состоянию оптимальное действие либо распределение вероятностей по доступным в нём действиям.

Методы на основе функции ценности

- Ещё один подход к решению задач RL.
- Здесь вместо обучения политики мы обучаем **функцию ценности**, сопоставляющую каждому состоянию ожидаемую ценность нахождения в нём.

Дополнения приветствуются!

Если вы хотите улучшить курс, можете создать Pull Request.

Этот глоссарий появился благодаря:

- @lucifermorningstar1305
- @daspartho
- @misza222

10 Обучаем первого агента глубокого обучения с подкреплением

Исходный ноутбук: https://colab.research.google.com/github/huggingface/deep-rl-class/blob/main/notebooks/unit1/unit1.ipynb

Теперь, когда вы изучили основы обучения с подкреплением, можно обучить первого агента и поделиться результатом с сообществом через Hub: лунный посадочный модуль научится правильно садиться на поверхность Луны.

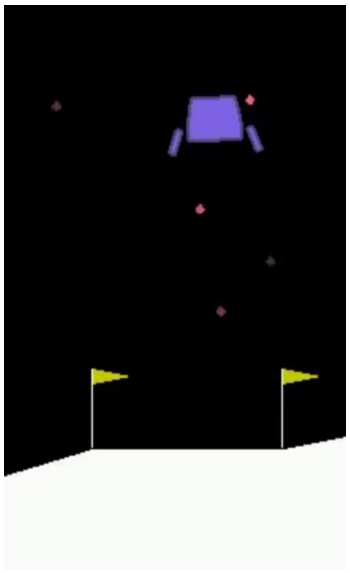


Рис. 39: Среда LunarLander: статичный кадр с официальной обложки курса; оригинальная GIF-анимация

И наконец, вы **загрузите обученного агента в Hugging Face Hub — бесплатную открытую платформу для обмена моделями, данными и демонстрациями.**

На нашей таблице лидеров вы сможете сравнить результаты с другими участниками и обменяться лучшими практиками повышения качества агента. Кто победит в соревновании первого модуля?

Чтобы это практическое задание было зачтено для получения сертификата, необходимо загрузить модель в Hub и **получить результат не ниже 200**.

Чтобы узнать свой результат, найдите модель в таблице лидеров; **результат = mean_reward - std вознаграждения**.

Если модели нет в списке, прокрутите страницу вниз и нажмите кнопку обновления.

Подробнее о сертификации: https://huggingface.co/deep-rl-course/en/unit0/introduction#certification-process

Отслеживать свой прогресс можно здесь: https://huggingface.co/spaces/ThomasSimonini/Check-my-progress-Deep-RL-Course

Приступим!

Чтобы начать практическое задание, нажмите Open In Colab:

Открыть исходный ноутбук в Google Colab

Мы настоятельно **рекомендуем студентам выполнять практические задания в Google Colab**, а не на личных компьютерах.

Благодаря Google Colab **вы можете сосредоточиться на обучении и экспериментах, не отвлекаясь на техническую настройку окружения.**

11 Модуль 1: обучаем первого агента Deep RL

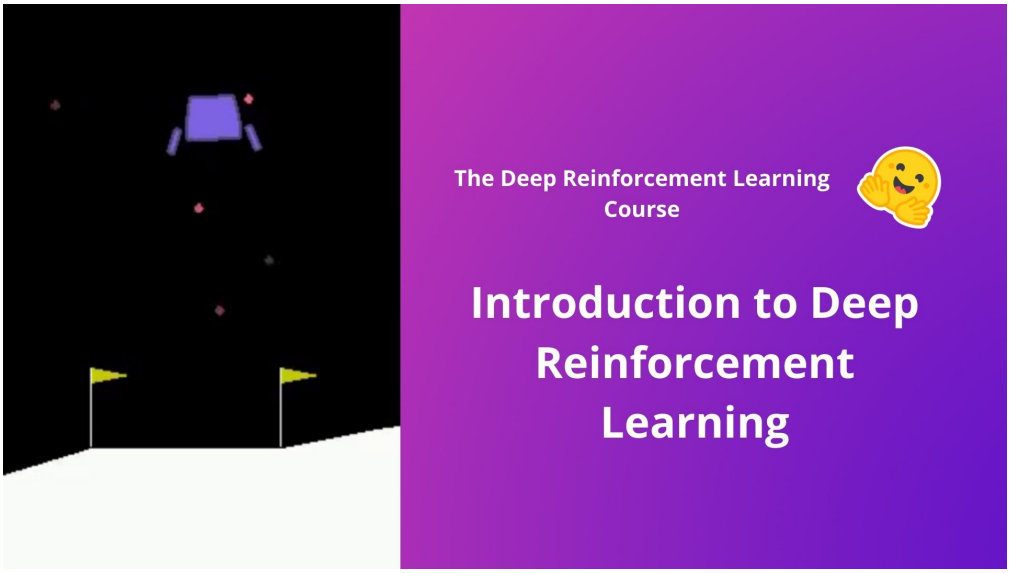


Рис. 40: Модуль 1: введение в глубокое обучение с подкреплением (официальная иллюстрация курса)

В этом ноутбуке вы обучите своего **первого агента глубокого обучения с подкреплением** — лунный посадочный модуль, который научится **правильно садиться на Луну**. Мы будем использовать библиотеку Stable-Baselines3, поделимся моделью с сообществом и поэкспериментируем с настройками.

Среда

- LunarLander-v2

Используемая библиотека

- Stable-Baselines3

Мы постоянно стараемся улучшать наши уроки. Если **вы обнаружили проблему в этом ноутбуке**, пожалуйста, создайте Issue в репозитории GitHub.

11.1 Цели ноутбука

После выполнения ноутбука вы сможете:

- Пользоваться **Gymnasium** — библиотекой сред.
- Пользоваться **Stable-Baselines3** — библиотекой глубокого обучения с подкреплением.
- **Публиковать обученного агента в Hub** вместе с видеозаписью его работы и оценкой качества.

11.2 Ноутбук курса Deep Reinforcement Learning



Рис. 41: Обзор курса Deep Reinforcement Learning

В этом бесплатном курсе вы:

- Изучите Deep RL **в теории и на практике**.
- Научитесь **использовать популярные библиотеки Deep RL**: Stable Baselines3, RL Baselines3 Zoo, CleanRL и Sample Factory 2.0.
- Обучите **агентов в необычных средах**.
- **Получите сертификат об окончании курса**, выполнив 80% заданий.

И это ещё не всё!

Программа курса: https://simoninithomas.github.io/deep-rl-course

Не забудьте **зарегистрироваться на курс** (мы собираем адреса электронной почты, чтобы **присылать ссылки по мере публикации модулей, а также информацию о заданиях и обновлениях**).

Лучший способ оставаться на связи и задавать вопросы — **присоединиться к нашему Discord-серверу**, чтобы общаться с сообществом и преподавателями: https://discord.gg/ydHrjt3WP5

11.3 Предварительные требования

Перед выполнением ноутбука необходимо:

Прочитать модуль 0, где собрана **основная информация о курсе и его организации**.

Разобраться в основах обучения с подкреплением (MC, TD, гипотеза вознаграждения и др.), прочитав модуль 1.

11.4 Краткое повторение Deep RL

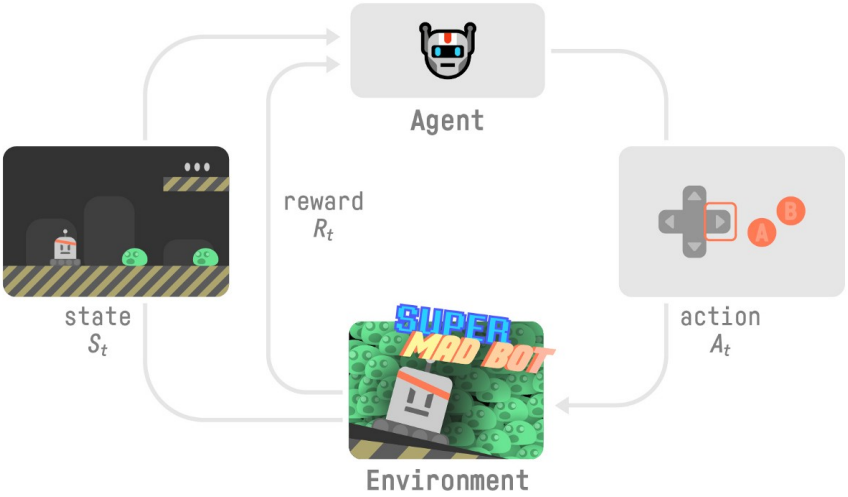


Рис. 42: Цикл взаимодействия в RL на примере видеоигры

Кратко повторим изученное в первом модуле:

- Обучение с подкреплением — **вычислительный подход к обучению на основе действий**. Мы создаём агента, который **взаимодействует со средой методом проб и ошибок** и получает вознаграждения (отрицательные или положительные) в качестве обратной связи.
- Цель любого RL-агента — **максимизировать ожидаемое суммарное вознаграждение** (ожидаемую отдачу), поскольку RL основано на *гипотезе вознаграждения*: любую цель можно описать как максимизацию ожидаемого суммарного вознаграждения.
- Процесс RL — **цикл, создающий последовательность состояний, действий, вознаграждений и следующих состояний**.
- Для расчёта ожидаемого суммарного вознаграждения **мы дисконтируем награды**: более близкие по времени вознаграждения вероятнее получить, поскольку их проще предсказать, чем далёкие будущие.
- Чтобы решить задачу RL, необходимо **найти оптимальную политику**; политика — «мозг» агента, указывающий действие для текущего состояния. Оптимальная политика максимизирует ожидаемую отдачу.

Оптимальную политику можно найти **двумя** способами:

- **Непосредственно обучать политику**: методы на основе политики.
- **Обучать функцию ценности**, оценивающую ожидаемую отдачу в каждом состоянии, и использовать её для построения политики: методы на основе функции ценности.
- Наконец, глубоким RL называют потому, что **для оценки выбираемого действия (методы на основе политики) или ценности состояния (методы на основе функции ценности) мы используем глубокие нейронные сети.**

12 Обучим первого агента Deep RL и загрузим его в Hub

12.1 Получение сертификата

Чтобы практическое задание было засчитано при сертификации, загрузите обученную модель в Hub и **получите результат не ниже 200**.

Чтобы узнать результат, найдите модель в таблице лидеров: **результат = mean_reward - стандартное отклонение вознаграждения**.

Дополнительная информация о сертификации: https://huggingface.co/deep-rl-course/en/unit0/introduction#certification-process

12.2 Настройка GPU

- Для **ускорения обучения агента воспользуемся GPU**. Для этого в Colab выберите `Runtime > Change Runtime type`.

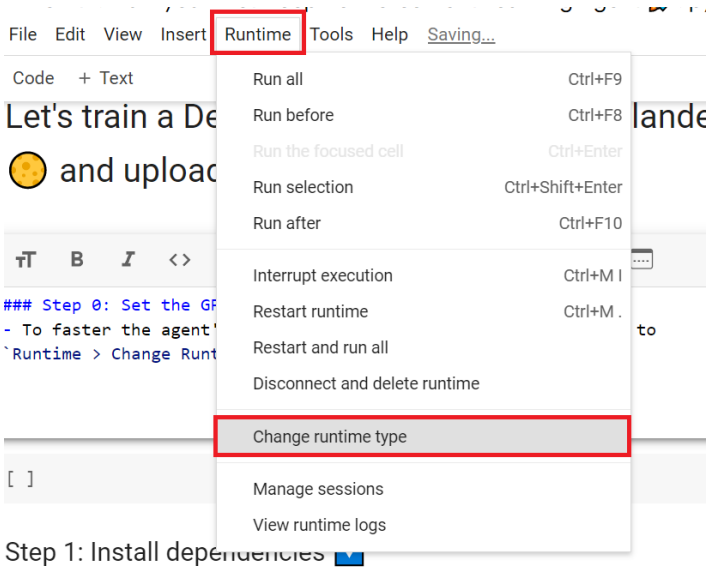


Рис. 43: Меню Google Colab для изменения типа среды выполнения

- Выберите `Hardware Accelerator > GPU`.

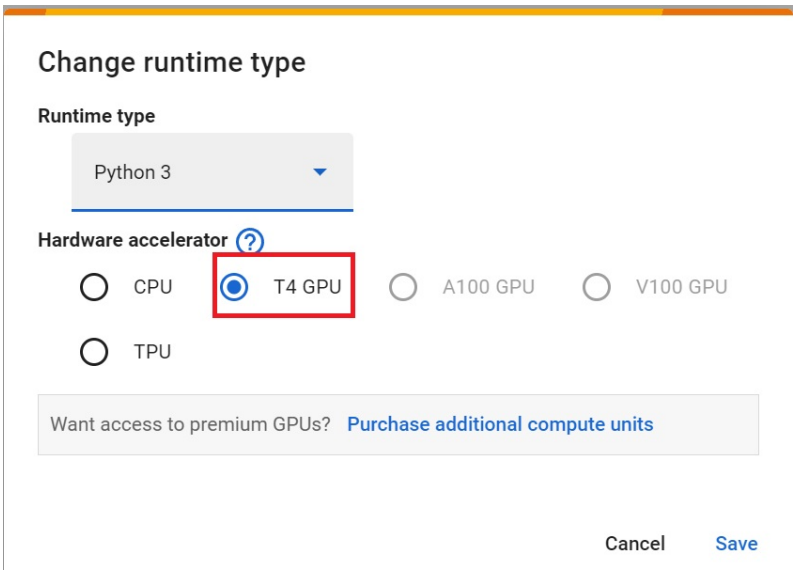


Рис. 44: Выбор аппаратного ускорителя GPU в Google Colab

12.3 Установка зависимостей и создание виртуального экрана

Сначала установим необходимые зависимости: нам потребуется несколько библиотек.

- `gymnasium[box2d]`: содержит среду LunarLander-v2.
- `stable-baselines3[extra]`: библиотека глубокого обучения с подкреплением.

- `huggingface_sb3`: дополнительные средства Stable-Baselines3 для загрузки моделей в Hugging Face Hub и скачивания их оттуда.

Чтобы упростить установку, мы создали скрипт, устанавливающий все эти зависимости.

```
apt install swig cmake
```

```
pip install -r https://raw.githubusercontent.com/huggingface/deep-rl-class/main/notebooks/unit1/
    requirements-unit1.txt
```

Во время работы с ноутбуком потребуется записать видео поведения агента. Для этого в Colab **нужно создать виртуальный экран, позволяющий визуализировать среду** и записывать кадры.

Поэтому следующая ячейка устанавливает библиотеки виртуального экрана, создаёт его и запускает.

```
sudo apt-get update
apt install python3-opengl
apt install ffmpeg
apt install xvfb
pip3 install pyvirtualdisplay
```

Чтобы новые библиотеки начали использоваться, **иногда необходимо перезапустить среду выполнения ноутбука**. Следующая ячейка намеренно вызовет **аварийную остановку среды, после чего понадобится снова подключиться и выполнять код, начиная с этого места**. Такой приём позволит запустить виртуальный экран.

```
import os

os.kill(os.getpid(), 9)
```

```
# Virtual display
from pyvirtualdisplay import Display

virtual_display = Display(visible=0, size=(1400, 900))
virtual_display.start()
```

12.4 Импорт библиотек

Дополнительно импортируем библиотеку huggingface_hub, **чтобы загружать обученные модели в Hub и скачивать их оттуда**.

Hugging Face Hub — единая платформа, где любой желающий может публиковать и изучать модели и наборы данных. Здесь есть версионирование, метрики, визуализации и другие функции, упрощающие совместную работу.

Посмотреть доступные модели глубокого обучения с подкреплением можно по ссылке: https://huggingface.co/models?pipeline_tag=reinforcement-learning&sort=downloads

```
import gymnasium

from huggingface_sb3 import load_from_hub, package_to_hub
from huggingface_hub import (
    notebook_login,
)  # To log to our Hugging Face account to be able to upload models to the Hub.
```

```
from stable_baselines3 import PPO
from stable_baselines3.common.env_util import make_vec_env
from stable_baselines3.common.evaluation import evaluate_policy
from stable_baselines3.common.monitor import Monitor
```

12.5 Как работает Gymnasium

Библиотека, содержащая нашу среду, называется Gymnasium. **При изучении глубокого RL вы будете часто пользоваться Gymnasium.**

Gymnasium — **новая версия библиотеки Gym**, поддерживаемая организацией Farama Foundation.

Gymnasium предоставляет две основные возможности:

- Интерфейс, позволяющий **создавать среды RL**.
- **Набор готовых сред** (gym-control, Atari, Box2D и т. д.).

Рассмотрим пример, но сначала вспомним цикл RL.

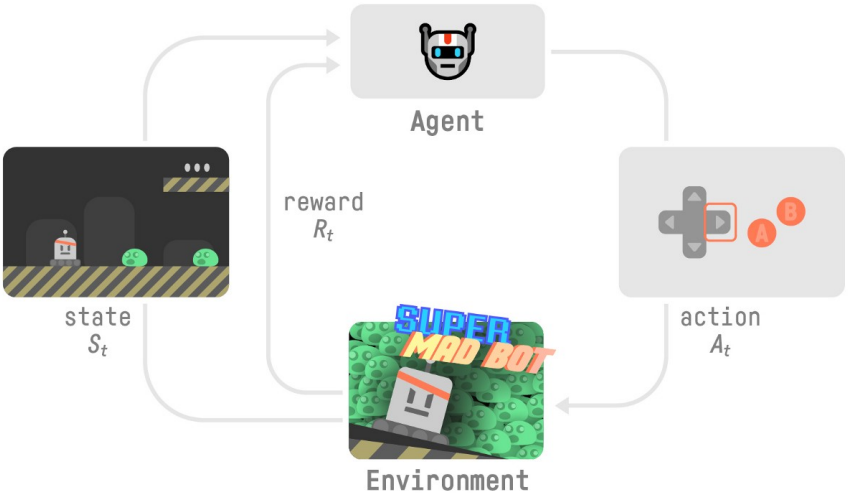


Рис. 45: Цикл взаимодействия в RL на примере видеоигры

На каждом шаге:

- Агент получает от **среды состояние (S0)** — первый кадр игры.
- На основе **состояния (S0)** агент выбирает **действие (A0)** — движение вправо.
- Среда переходит в **новое состояние (S1)** — следующий кадр.
- Среда выдаёт агенту **вознаграждение (R1)** — мы не погибли, то есть получили *положительное вознаграждение +1*.

В Gymnasium это выглядит так:

1 Создаём среду с помощью `gymnasium.make()`.

2 Сбрасываем среду в начальное состояние командой `observation = env.reset()`.

На каждом шаге:

3 Получаем действие от модели (в нашем примере выбирается случайное действие).

4 С помощью `env.step(action)` выполняем действие в среде и получаем:

- `observation`: новое состояние (st+1).
- `reward`: вознаграждение после выполнения действия.

- `terminated`: признак завершения эпизода (агент достиг терминального состояния).
- `truncated`: новый для этой версии признак обрыва эпизода, например по ограничению времени или при выходе агента за границы среды.
- `info`: словарь с дополнительной информацией (зависит от среды).

Подробнее см. https://gymnasium.farama.org/api/env/#gymnasium.Env.step

Если эпизод завершился:

- Сбрасываем среду в начальное состояние командой `observation = env.reset()`.

Рассмотрим пример! Обязательно прочитайте код.

```
import gymnasium as gym

# First, we create our environment called LunarLander-v2
env = gym.make("LunarLander-v2")

# Then we reset this environment
observation, info = env.reset()

for _ in range(20):
    # Take a random action
    action = env.action_space.sample()
    print("Action taken:", action)

    # Do this action in the environment and get
    # next_state, reward, terminated, truncated and info
    observation, reward, terminated, truncated, info = env.step(action)

    # If the game is terminated (in our case we land, crashed) or truncated (timeout)
    if terminated or truncated:
        # Reset the environment
        print("Environment is reset")
        observation, info = env.reset()

env.close()
```

12.6 Создание среды LunarLander и знакомство с ней

Среда

В этом первом руководстве мы научим агента Lunar Lander **правильно садиться на Луну**. Для этого он должен научиться **управлять скоростью и положением (по горизонтали, вертикали и углу наклона), чтобы выполнить безопасную посадку.**

При работе с новой средой полезно сначала изучить её документацию:

https://gymnasium.farama.org/environments/box2d/lunar_lander/

Посмотрим, как выглядит эта среда:

```
# We create our environment with gym.make("<name_of_the_environment>")
env = gym.make("LunarLander-v2")
env.reset()
print("_____OBSERVATION SPACE_____ \n")
```

```
print("Observation Space Shape", env.observation_space.shape)
print("Sample observation", env.observation_space.sample())  # Get a random observation
```

Из строки `Observation Space Shape (8,)` видно, что наблюдение — вектор из 8 элементов, каждый из которых содержит определённую информацию о посадочном модуле:

- Горизонтальная координата (x).
- Вертикальная координата (y).
- Горизонтальная скорость (x).
- Вертикальная скорость (y).
- Угол наклона.
- Угловая скорость.
- Признак касания земли левой опорой (логический).
- Признак касания земли правой опорой (логический).

```
print("\n _____ACTION SPACE_____ \n")
print("Action Space Shape", env.action_space.n)
print("Action Space Sample", env.action_space.sample())  # Take a random action
```

Пространство действий (множество доступных агенту действий) дискретно и состоит из четырёх вариантов:

- Действие 0: ничего не делать.
- Действие 1: включить левый боковой двигатель ориентации.
- Действие 2: включить главный двигатель.
- Действие 3: включить правый боковой двигатель ориентации.

Функция вознаграждения (выдающая награду на каждом временном шаге):

После каждого шага агент получает вознаграждение. Суммарное вознаграждение за эпизод — **сумма вознаграждений за все шаги этого эпизода**.

На каждом шаге вознаграждение:

- Увеличивается или уменьшается по мере приближения или удаления от посадочной площадки.
- Увеличивается при замедлении и уменьшается при ускорении посадочного модуля.
- Уменьшается при увеличении наклона модуля (отклонения от горизонтали).
- Увеличивается на 10 баллов за каждую опору, касающуюся земли.
- Уменьшается на 0,03 балла за каждый кадр работы бокового двигателя.
- Уменьшается на 0,3 балла за каждый кадр работы главного двигателя.

Кроме того, эпизод приносит **дополнительное вознаграждение -100 при аварии или +100 при успешной посадке**.

Эпизод **считается успешным, если итоговый результат составляет не менее 200 баллов**.

Векторизованная среда

- Создадим векторизованную среду (способ объединить несколько независимых сред в одну) из 16 экземпляров. Так **во время обучения агент получит более разнообразный опыт**.

```
# Create the environment
env = make_vec_env("LunarLander-v2", n_envs=16)
```

12.7 Создание модели

- Мы изучили среду и поняли задачу: **научиться безопасно сажать LunarLander на посадочную площадку, управляя левым и правым боковыми и главным двигателями**. Теперь построим алгоритм решения.
- Для этого используем первую библиотеку Deep RL в курсе — Stable Baselines3 (SB3).
- SB3 — набор **надёжных реализаций алгоритмов обучения с подкреплением на PyTorch**.

При знакомстве с новой библиотекой полезно сначала изучить документацию, а затем выполнить несколько руководств.



Рис. 46: Иллюстрация библиотеки Stable-Baselines3

Для решения задачи воспользуемся алгоритмом **PPO** из SB3. PPO (Proximal Policy Optimization) — один из современных алгоритмов глубокого обучения с подкреплением, которые будут рассмотрены в этом курсе.

В PPO объединяются:

- *Метод RL на основе функции ценности*: обучение функции ценности действия, показывающей **наиболее ценное действие для заданной пары состояния и действия**.
- *Метод RL на основе политики*: обучение политики, **возвращающей распределение вероятностей по действиям**.

Настроить Stable-Baselines3 достаточно просто:

1 **Создайте среду** (в нашем случае это уже сделано выше).

2 **Выберите модель и создайте её экземпляр**: `model = PPO("MlpPolicy")`.

3 **Обучите агента** через `model.learn`, задав число временных шагов обучения.

```
# Create environment
env = gym.make('LunarLander-v2')

# Instantiate the agent
```

```
model = PPO('MlpPolicy', env, verbose=1)
# Train the agent
model.learn(total_timesteps=int(2e5))
```

```
# TODO: Define a PPO MlpPolicy architecture
# We use MultiLayerPerceptron (MLPPolicy) because the input is a vector,
# if we had frames as input we would use CnnPolicy
model =
```

Решение

```
# SOLUTION
# We added some parameters to accelerate the training
model = PPO(
    policy="MlpPolicy",
    env=env,
    n_steps=1024,
    batch_size=64,
    n_epochs=4,
    gamma=0.999,
    gae_lambda=0.98,
    ent_coef=0.01,
    verbose=1,
)
```

12.8 Обучение агента PPO

- Обучим агента в течение 1 000 000 шагов; не забудьте включить GPU в Colab. Это займёт примерно ~20 минут, но для первого эксперимента можно использовать меньше шагов.
- Пока идёт обучение, сделайте заслуженный перерыв!

```
# TODO: Train it for 1,000,000 timesteps
# TODO: Specify file name for model and save the model to file
model_name = "ppo-LunarLander-v2"
```

Решение

```
# SOLUTION
# Train it for 1,000,000 timesteps
model.learn(total_timesteps=1000000)
# Save the model
model_name = "ppo-LunarLander-v2"
model.save(model_name)
```

12.9 Оценка качества агента

- Не забудьте обернуть среду в Monitor.
- Теперь, когда агент LunarLander обучен, нужно **проверить качество его работы**.
- Для этого Stable-Baselines3 предоставляет функцию `evaluate_policy`.
- Чтобы заполнить этот раздел, обратитесь к документации.
- Далее мы узнаем, **как автоматически оценивать и публиковать агента для соревнования в таблице лидеров, но сначала сделаем оценку вручную**.

При оценке качества следует создавать отдельную среду для тестирования, а не использовать обучающую.

```
# TODO: Evaluate the agent
# Create a new environment for evaluation
eval_env =

# Evaluate the model with 10 evaluation episodes and deterministic=True
mean_reward, std_reward =

# Print the results
```

Решение

```
# @title
eval_env = Monitor(gym.make("LunarLander-v2"))
mean_reward, std_reward = evaluate_policy(model, eval_env, n_eval_episodes=10, deterministic=True)
print(f"mean_reward={mean_reward:.2f} +/- {std_reward}")
```

- В моём случае после миллиона шагов среднее вознаграждение составило `200.20 +/- 20.80`. Это означает, что наш лунный модуль готов к посадке на Луну.

12.10 Публикация обученной модели в Hub

Теперь, когда мы получили хорошие результаты, можно опубликовать обученную модель в Hub одной строкой кода.

Документация библиотеки: https://github.com/huggingface/huggingface_sb3/tree/main#hugging-face--x-stable-baselines3-v20

Вот пример карточки модели (для Space Invaders):

С помощью `package_to_hub` **можно оценить агента, записать видео его работы, создать карточку модели и загрузить всё в Hub**.

Таким образом:

- Вы можете **демонстрировать свою работу**.
- Вы можете **наблюдать за тем, как играет агент**.
- Вы можете **поделиться с сообществом агентом, которого смогут использовать другие**.
- Вы можете **сравнивать качество своего агента с результатами других участников в таблице лидеров**: https://huggingface.co/spaces/huggingface-projects/Deep-Reinforcement-Learning-Leaderboard

Чтобы опубликовать модель для сообщества, нужно выполнить ещё три шага:

1 Если ещё не сделали этого, создайте аккаунт Hugging Face: https://huggingface.co/join

2 Войдите в аккаунт и сохраните токен аутентификации с сайта Hugging Face.

- Создайте новый токен (https://huggingface.co/settings/tokens) **с правом записи (write)**.

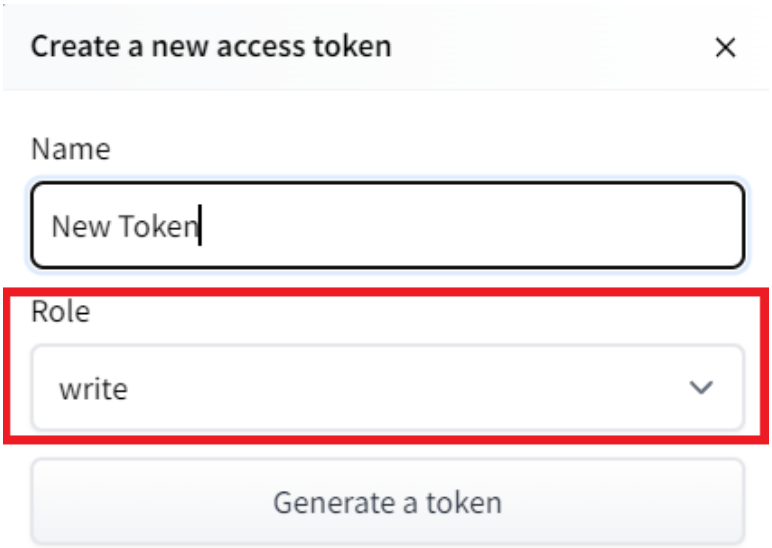


Рис. 47: Создание токена Hugging Face Hub с правами на запись

- Скопируйте токен.
- Выполните ячейку ниже и вставьте токен.

```
notebook_login()
!git config --global credential.helper store
```

Если вы не используете Google Colab или Jupyter Notebook, вместо этого выполните команду `huggingface-cli login`.

3 Теперь всё готово к публикации обученного агента в Hub через функцию `package_to_hub()`.

Заполним параметры функции `package_to_hub`:

- `model`: наша обученная модель.
- `model_name`: имя обученной модели, которое мы задали в `model_save`.
- `model_architecture`: архитектура модели, в нашем случае PPO.
- `env_id`: название среды, в нашем случае `LunarLander-v2`.
- `eval_env`: среда оценки, заданная переменной eval_env.
- `repo_id`: имя репозитория Hugging Face Hub, который будет создан или обновлён, `(repo_id = {username}/{repo_name})`.

Подходящий формат имени — `{username}/{model_architecture}-{env_id}`.

- `commit_message`: сообщение коммита.

```
import gymnasium as gym
from stable_baselines3.common.vec_env import DummyVecEnv
from stable_baselines3.common.env_util import make_vec_env

from huggingface_sb3 import package_to_hub

## TODO: Define a repo_id
## repo_id is the id of the model repository from the Hugging Face Hub (repo_id = {organization}/{
    repo_name} for instance ThomasSimonini/ppo-LunarLander-v2
repo_id =

# TODO: Define the name of the environment
env_id =

# Create the evaluation env and set the render_mode="rgb_array"
```

```
eval_env = DummyVecEnv([lambda: gym.make(env_id, render_mode="rgb_array")])


# TODO: Define the model architecture we used
model_architecture = ""

## TODO: Define the commit message
commit_message = ""

# method save, evaluate, generate a model card and record a replay video of your agent before
    pushing the repo to the hub
package_to_hub(model=model, # Our trained model
               model_name=model_name, # The name of our trained model
               model_architecture=model_architecture, # The model architecture we used: in our case
     PPO
               env_id=env_id, # Name of the environment
               eval_env=eval_env, # Evaluation Environment
               repo_id=repo_id, # id of the model repository from the Hugging Face Hub (repo_id = {
    organization}/{repo_name} for instance ThomasSimonini/ppo-LunarLander-v2
               commit_message=commit_message)
```

Решение

```
import gymnasium as gym

from stable_baselines3 import PPO
from stable_baselines3.common.vec_env import DummyVecEnv
from stable_baselines3.common.env_util import make_vec_env

from huggingface_sb3 import package_to_hub

# PLACE the variables you've just defined two cells above
# Define the name of the environment
env_id = "LunarLander-v2"

# TODO: Define the model architecture we used
model_architecture = "PPO"

## Define a repo_id
## repo_id is the id of the model repository from the Hugging Face Hub (repo_id = {organization}/{
    repo_name} for instance ThomasSimonini/ppo-LunarLander-v2
## CHANGE WITH YOUR REPO ID
repo_id = "ThomasSimonini/ppo-LunarLander-v2"  # Change with your repo id, you can't push with mine
     :)

## Define the commit message
commit_message = "Upload PPO LunarLander-v2 trained agent"

# Create the evaluation env and set the render_mode="rgb_array"
eval_env = DummyVecEnv([lambda: Monitor(gym.make(env_id, render_mode="rgb_array"))])

# PLACE the package_to_hub function you've just filled here
package_to_hub(
    model=model,  # Our trained model
    model_name=model_name,  # The name of our trained model
    model_architecture=model_architecture,  # The model architecture we used: in our case PPO
    env_id=env_id,  # Name of the environment
    eval_env=eval_env,  # Evaluation Environment
    repo_id=repo_id,  # id of the model repository from the Hugging Face Hub (repo_id = {
    organization}/{repo_name} for instance ThomasSimonini/ppo-LunarLander-v2
    commit_message=commit_message,
)
```

Поздравляем! Вы обучили и опубликовали своего первого агента глубокого RL. Скрипт выше должен вывести ссылку на репозиторий модели, например https://huggingface.co/osanseviero/test_sb3. На этой странице можно:

- Посмотреть справа видеопревью работы агента.
- Перейти в раздел «Files and versions», чтобы увидеть все файлы репозитория.
- Нажать «Use in stable-baselines3», чтобы получить фрагмент кода для загрузки модели.
- Прочитать карточку модели (файл `README.md`), содержащую описание модели.

В основе Hub лежат репозитории Git (не страшно, если вы пока не знакомы с Git). Это позволяет загружать новые версии модели по мере экспериментов и улучшения качества агента.

Сравните результаты LunarLander-v2 с результатами других студентов с помощью таблицы лидеров.

12.11 Загрузка сохранённой модели LunarLander из Hub

Благодарим ironbar за вклад в материалы.

Загрузить сохранённую модель из Hub очень просто.

На странице https://huggingface.co/models?library=stable-baselines3 можно увидеть все сохранённые модели Stable-Baselines3.

1. Выберите одну из них и скопируйте её repo_id.

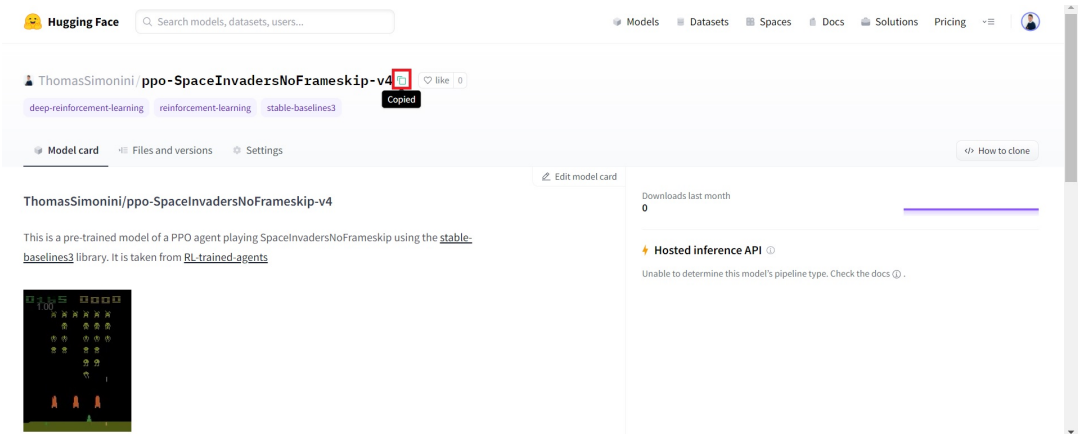


Рис. 48: Копирование идентификатора репозитория модели в Hugging Face Hub

2. Затем используйте load_from_hub с параметрами:

- Идентификатор repo_id.
- Имя файла: сохранённая модель в репозитории с расширением (*.zip).

Поскольку модель, которую я загружаю из Hub, была обучена с Gym (старой версией Gymnasium), нужно установить Shimmy — средство преобразования API, позволяющее корректно запустить среду.

Документация Shimmy: https://github.com/Farama-Foundation/Shimmy

```
!pip install shimmy
```

```
from huggingface_sb3 import load_from_hub

repo_id = "Classroom-workshop/assignment2-omar"  # The repo_id
filename = "ppo-LunarLander-v2.zip"  # The model filename.zip
```

```
# When the model was trained on Python 3.8 the pickle protocol is 5
# But Python 3.6, 3.7 use protocol 4
# In order to get compatibility we need to:
# 1. Install pickle5 (we done it at the beginning of the colab)
# 2. Create a custom empty object we pass as parameter to PPO.load()
custom_objects = {
    "learning_rate": 0.0,
    "lr_schedule": lambda _: 0.0,
    "clip_range": lambda _: 0.0,
}

checkpoint = load_from_hub(repo_id, filename)
model = PPO.load(checkpoint, custom_objects=custom_objects, print_system_info=True)
```

Проверим качество этого агента:

```
# @title
eval_env = Monitor(gym.make("LunarLander-v2"))
mean_reward, std_reward = evaluate_policy(model, eval_env, n_eval_episodes=10, deterministic=True)
print(f"mean_reward={mean_reward:.2f} +/- {std_reward}")
```

12.12 Дополнительные задания

Лучший способ учиться — **экспериментировать самостоятельно**! Как вы видели, текущий агент работает не слишком хорошо. Сначала попробуйте увеличить число шагов обучения. При 1 000 000 шагов нам удалось получить отличные результаты!

Своих агентов можно найти в таблице лидеров. Сможете подняться на вершину рейтинга?

Вот несколько идей, как добиться этого:

- Увеличьте число шагов обучения.
- Попробуйте другие гиперпараметры `PPO`; список приведён в https : / / stable-baselines3 . readthedocs.io/en/master/modules/ppo.html#parameters.
- Ознакомьтесь с документацией Stable-Baselines3 и попробуйте другую модель, например DQN.
- **Загрузите новую обученную модель** в Hub.

Сравните результаты своего LunarLander-v2 с результатами других студентов через таблицу лидеров.

Посадка на Луну кажется слишком скучной? Попробуйте **сменить среду**: например, MountainCar-v0, CartPole-v1 или CarRacing-v0. Изучите их в документации Gym и экспериментируйте!

Поздравляем с завершением главы! Это была самая большая глава курса, **и информации действительно было много.**

Если какие-то понятия всё ещё вызывают затруднения, это совершенно нормально! **У меня и у всех изучавших RL было то же самое.**

Прежде чем двигаться дальше, постарайтесь **хорошо усвоить материал и выполнить дополнительные задания**. Важно освоить эти основы и приобрести прочную базу.

Конечно, далее мы будем изучать эти понятия подробнее, но **лучше разобраться с ними уже сейчас, прежде чем переходить к следующим главам**.

В следующем, первом бонусном модуле, вы научите пса Huggy приносить палку.



Рис. 49: Huggy в виртуальной обучающей среде

12.13 Продолжайте учиться, у вас всё получится

13 Тест для самопроверки

Лучший способ учиться и не поддаваться иллюзии компетентности — **проверять себя**. Это поможет понять, **какие знания нужно закрепить**.

Вопрос 1: что такое обучение с подкреплением?

Вопрос 2: опишите цикл RL

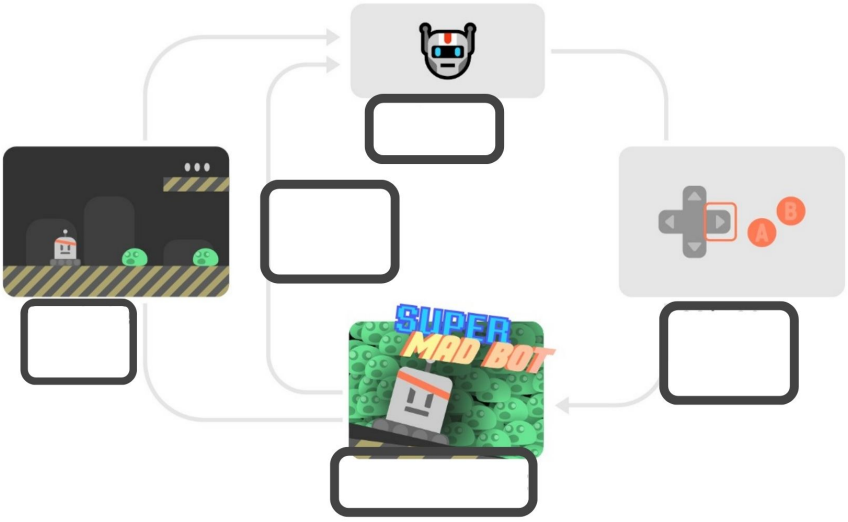


Рис. 50: Задание теста: впишите состояние, действие, следующее состояние и вознаграждение в цикл RL

На каждом шаге:

- Агент получает от среды _____.
- На основе _____ агент выбирает _____.
- Агент движется вправо.
- Среда переходит в _____.
- Среда выдаёт агенту _____.

Выберите все правильные варианты (их может быть несколько).

- **A.** действие a0, действие a0, состояние s0, состояние s1, вознаграждение r1.
- **B.** состояние s0, состояние s0, действие a0, новое состояние s1, вознаграждение r1.
- **C.** состояние s0, состояние s0, действие a0, состояние s1, действие a1.

Вопрос 3: чем состояние отличается от наблюдения?

Выберите все правильные варианты (их может быть несколько).

- **A.** Состояние полностью описывает мир (скрытой информации нет).
- **B.** Состояние представляет собой частичное описание мира.
- **C.** Наблюдение полностью описывает мир (скрытой информации нет).
- **D.** Наблюдение представляет собой частичное описание состояния.
- **E.** В шахматной среде мы получаем состояние.
- **F.** В шахматной среде мы получаем наблюдение.
- **G.** В Super Mario Bros мы получаем состояние.
- **H.** В Super Mario Bros мы получаем наблюдение.

Вопрос 4: какие существуют два типа задач RL?

Выберите все правильные варианты (их может быть несколько).

- **A.** Эпизодические.
- **B.** Рекурсивные.
- **C.** Состязательные.
- **D.** Продолжающиеся.

Вопрос 5: в чём состоит компромисс между исследованием и использованием знаний?

Вопрос 6: что такое политика?

Вопрос 7: что такое методы на основе функции ценности?

Вопрос 8: что такое методы на основе политики?

13.1 Ответы и объяснения

Вопрос 1: что такое обучение с подкреплением?

Обучение с подкреплением — это **подход к решению задач управления (принятия решений)**, при котором создаются агенты, обучающиеся взаимодействовать со средой методом проб и ошибок и **получающие вознаграждения (положительные или отрицательные) в качестве единственной обратной связи**.

Вопрос 2: опишите цикл RL

Ответ: B.

- **Вариант A:** На каждом шаге агент получает от среды **состояние s0**. По **состоянию s0** он выбирает **действие a0**, например движение вправо. Среда переходит в **новое состояние s1** и выдаёт агенту **вознаграждение r1**.
- **Вариант C:** На каждом шаге агент получает от среды **состояние s0**. По **состоянию s0** он выбирает **действие a0**, например движение вправо. Среда переходит в **новое состояние s1** и выдаёт агенту **вознаграждение r1**.

Вопрос 3: чем состояние отличается от наблюдения?

Ответ: A, D, E, H.

- **Вариант E (верный):** Нам известна вся информация о шахматной доске.
- **Вариант F:** Нам известна вся информация о шахматной доске.
- **Вариант G:** Мы видим только часть уровня рядом с персонажем, поэтому получаем наблюдение.

- **Вариант H (верный):** Мы видим только часть уровня рядом с персонажем.

Вопрос 4: какие существуют два типа задач RL?

Ответ: A, D.

- **Вариант A (верный):** Эпизодическая задача имеет начало и конец (терминальное состояние). Это образует эпизод — последовательность состояний, действий, вознаграждений и следующих состояний. Например, в Super Mario Bros эпизод начинается с запуска нового уровня и завершается, когда персонаж погибает или доходит до конца.
- **Вариант D (верный):** Продолжающиеся задачи идут бесконечно (терминального состояния нет). В таких задачах агент должен учиться выбирать оптимальные действия, одновременно продолжая взаимодействовать со средой.

Вопрос 5: в чём состоит компромисс между исследованием и использованием знаний?

В RL необходимо **соблюдать баланс между изучением среды и использованием уже известных сведений о ней**.

- *Исследование (exploration)* — изучение среды с помощью **случайных действий для получения дополнительной информации**.
- *Использование знаний (exploitation)* — **применение известной информации для максимизации вознаграждения**.

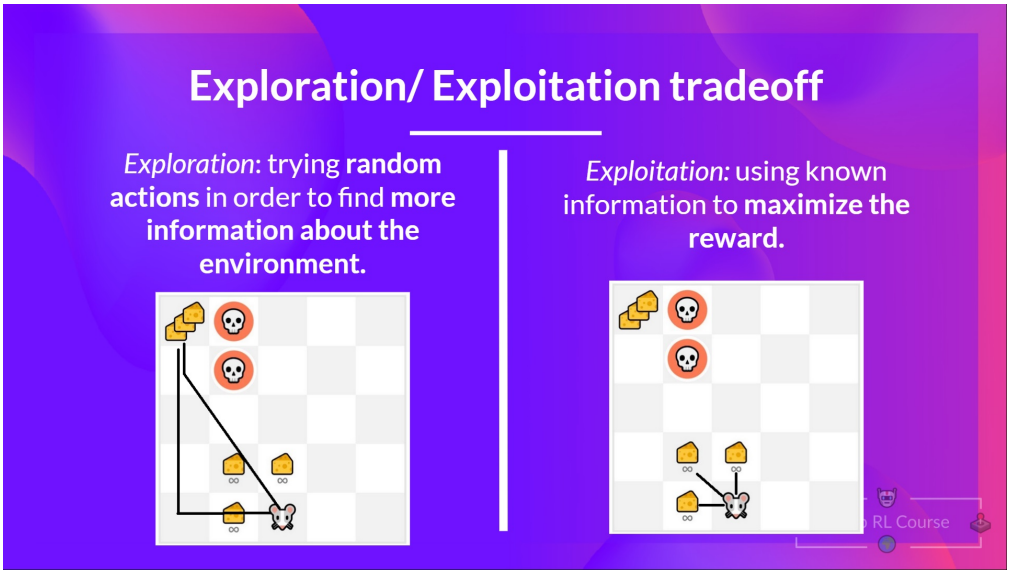


Рис. 51: Исследование и использование знаний: обобщающая схема

Вопрос 6: что такое политика?

- Политика π — **«мозг» агента**. Она определяет действие, которое следует выбрать в текущем состоянии, а значит, задаёт поведение агента в каждый момент времени.

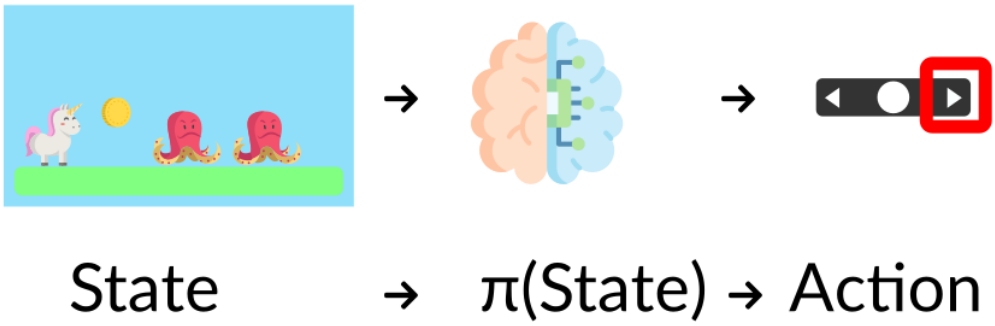


Рис. 52: Политика отображает текущее состояние в действие

Вопрос 7: что такое методы на основе функции ценности?

- Методы на основе функции ценности — один из основных подходов к решению задач RL.
- Вместо непосредственного обучения политики **мы обучаем функцию ценности, сопоставляющую состоянию ожидаемую ценность нахождения в нём**.

Вопрос 8: что такое методы на основе политики?

- В *методах на основе политики* мы **непосредственно обучаем функцию политики**.
- Такая функция **сопоставляет каждому состоянию наиболее подходящее действие** или **распределение вероятностей по доступным действиям**.

Поздравляем с прохождением теста! Если какие-то темы вызвали затруднения, ещё раз прочитайте соответствующие разделы для закрепления материала. Но **не переживайте**: в курсе мы неоднократно вернёмся к этим понятиям и **закрепим теорию практическими заданиями**.

14 Заключение

Поздравляем с завершением модуля! **Это был самый объёмный модуль**, и мы рассмотрели много новой информации. Поздравляем и с выполнением практической части: вы только что обучили первых агентов Deep RL и поделились ими с сообществом!

Если некоторые понятия пока кажутся запутанными, это нормально. Так было и у меня, и у всех, кто начинал изучать RL.

Не торопитесь: прежде чем двигаться дальше, постарайтесь как следует освоить материал. Важно иметь прочную базу перед переходом к более увлекательным задачам.

Конечно, в ходе курса мы будем снова использовать и объяснять эти термины, но лучше понять их до перехода к следующим модулям.

В следующем (бонусном) модуле мы закрепим пройденное, **обучив пса Huggy приносить палку**.

После этого вы сможете с ним поиграть.



Рис. 53: Пёс Huggy: обучающая среда из бонусного модуля

Наконец, мы были бы рады **узнать ваше мнение о курсе и о том, как его можно улучшить**. Если хотите поделиться отзывом, пожалуйста, заполните анкету.

Продолжайте учиться, у вас всё получится

15 Дополнительная литература

Эти материалы **необязательны**, но помогут глубже разобраться в теме.

15.1 Глубокое обучение с подкреплением

- Reinforcement Learning: An Introduction, Richard Sutton и Andrew G. Barto, главы 1, 2 и 3
- Серия Foundations of Deep RL: лекция 1 о MDP, точных методах решения и RL с максимальной энтропией, Pieter Abbeel
- Spinning Up RL от OpenAI, часть 1: ключевые понятия RL

15.2 Gym

- Начало работы с OpenAI Gym: основные строительные блоки
- Создание собственной среды Gym